ADVANTEST

Time Measurement Unit

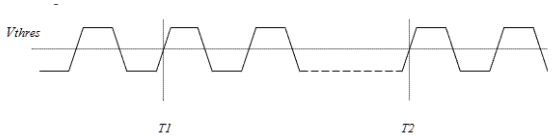
Objectives

- Provide instruction on how to use TMU instrument to make various types of tests

- Provide example program demonstrating the concepts
  - Requirements
    - SMT 7.5.2.3
    - PS1600 channel card for online use
    - No HIB required to make measurements

Overview of Time Measurement Unit (TMU) per pin

- The TMU (Time Measurement Unit) per pin is integrated into the test processor of the Smart Scale cards
- Capability to accurately measure the period (frequency), jitter, pulse width, and rise/fall time of periodic and non-periodic output signals
  - The TMU returns the time from the point in time when the TMU is *armed* to the captured event
  - Events are defined by a slope (rising or falling)
  - Events are captured when it crosses a voltage threshold
- TMU measurements can be done on several pins in parallel
  - Parallel pin measurements can be setup differently



References and Additional Information:
Concept 125212

- The TMU (Time Measurement Unit) per pin is integrated into the test processor of the Smart Scale cards. It provides a fast and easy way to accurately measure the period (frequency), jitter, pulse width, and rise/fall time of periodic and non-periodic output signals.
- The TMU measures the distance of a certain type of *event* from the point in time when the TMU is *armed* (activated). Events are defined by a slope (rising or falling) of the DUT signal to be measured, and by a voltage threshold. The event occurs whenever a slope of the selected type crosses the specified voltage threshold. If the event is defined by a rising (falling) slope, then only rising (falling) slopes will be taken into consideration. But you can also require that alternately rising and falling slopes are considered as events. This option allows you to determine the distance between a rising and a falling slope in one measurement. The activation (arming) of the TMU is controlled from the pattern by special waveforms
- Since every channel has its own TMU, it is possible to make TMU measurements on several pins in parallel. These parallel measurements can be different from pin to pin. For example, the period length can be measured on

one pin, and jitter on another. You can also execute a functional test for a receive pin and analyze its output signal with the TMU in parallel. A PLL can be kept alive during a TMU measurement.

- **Note** The TMU on the Pin Scale 1600 card can also be used on the high voltage channels

PS1600 TMU Specifications

Time Measurement Unit - TMU per pin

- **DC performance -** Specifications of receiver apply.
- **AC performance**

AC performance	
Maximum input frequency	800 MHz
Linearity error	±5 ps
Accuracy, single timestamp to ARM (time zero)	±35 ps
Accuracy, skew/propagation delay (between different pin resources)	2 * (EPA + 35 ps)
Intrinsic jitter (RMS)	5 ps
Resolution	1 ps
Other (maximum number of simultaneously active TMUs	170 per 4 slot segment)
Maximum sampling rate	50 Msps
Measurement mode	TMU measurements of single ended or differential compared signals

EPA:

- *TDC Topic 126225*
- *System EPA <+/- 100ps*
- *Board EPA < +/-80ps*

References and Additional Information:
> Pin Scale 1600 digital card (Smart Scale) > Pin Scale 1600 TMU specifications: TDC Reference 126239
Receiver specs Reference 126232

For EPA referenced in slide Here is the PS1600 EPA Specification:
TDC Reference 126225 **OTA and EPA**

System OTA and EPA	
Overall timing accuracy (t_{OTA})	< ±200 ps
Edge placement accuracy ($t_{EPA}=t_{OTA}/2$)	< ±100 ps
Per-board OTA and EPA	
Overall timing accuracy per board (t_{OTA})	< ±160 ps
Edge placement accuracy per board ($t_{EPA}=t_{OTA}/2$)	< ±80 ps
Edge placement	
Edge placement resolution	1 ps
Edge placement range[1)]	-4 to 32 sequencer periods

[1)] Edge placement range maximal -6300 ns to +19000 ns. For sequencer period >752

ns the edge placement range is -4 to 12 sequencer periods

Sampling Rate

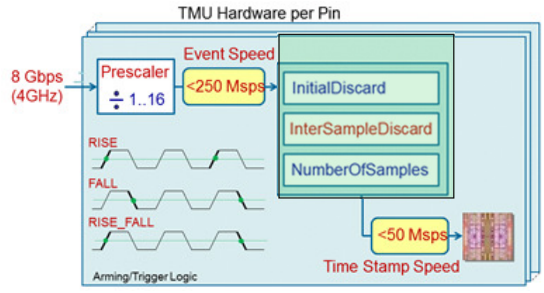
- Maximum sampling rate 50 Msps
- The operation of the TMU is determined by its *sample rate*.
  - This is the minimum distance that must exist between two samples, that is, measured events.
  - If the event to be measured can occur at a shorter distance, the DUT signal must be transformed in order to be correctly measurable.
  - This transformation is performed by two initial processing blocks of the TMU, a *prescaler* and an *event discarding block* (in this order).

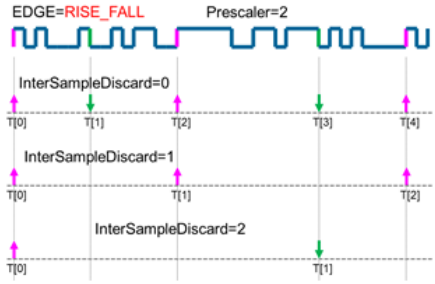
References and Additional Information:
Concept 125212

20nS sampling (50Msps)

Hardware Settings for Preconditioning Input Signal to < 50Msps

- Prescaler divides input down before event discarding block.
- Event discarding block uses InterSampleDiscard/Interskip and initial discard to limit number of samples to < 50Msps





MAPI::
Diagram shows the MAPI interSampleDiscard. SmartRDI would be interSkip.
Prescaler divides input down before event discarding block.
Event discarding block is shown in green shading.
Event discarding block uses InterSampleDiscard/Interskip and initial discard to limit number of samples to < 50Msps

References and Additional Information:
Topic 144892 PRBS Jitter Measurement by TMU

Parameters for TMU Tests

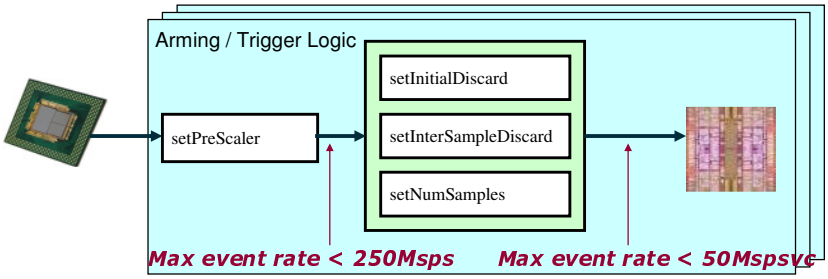
- Input Data Conditioning
  - Prescaler
  - Interskip
  - InitDiscard
  - DataRate

- Additional Parameters
  - Vth (voltage threshold) Single Threshold statement (RISE and FALL time shows how to do 2 tests)
  - Samples
  - VectorOffset
  - Capture Edge type (Edge Select : RISE, FALL, RISE_FALL)

References and Additional Information:

TMU per Pin - Concepts

PS1600
PS9G



TMU specifics:

The incoming data rate has to be scaled to meet the limitations at the input stages. The user has the possibility to:

- Specify the in incoming data rate and have the SW calculate the Pre-Scaler and Discard stages.
- Specify the Pre-Scaler and Discard stage manually

MAPI::
interSampleDiscard for MAPI is same as interskip for RDI

Prescaler

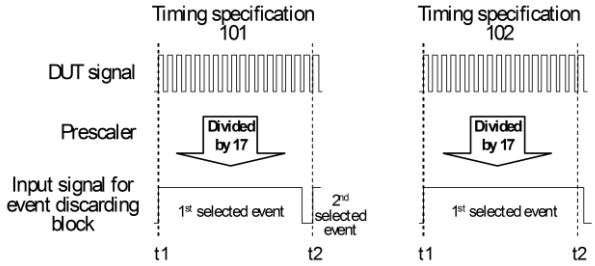
- Prescaler can be used for dealing with very high data rates to condition signal within the 50Msps spec
- Prescaler is a frequency divider up to 1/16 to condition the signal prior to the event discarding hardware block
- The prescaler reduces the frequency of the DUT signal by a user definable factor
- Prescaler value greater than 1 can suppress input signal events
  - Set value to 1 for no prescaling
  - To Suppress edges: Prescaler Values 2-16 (17 for RISE_FALL)

- The input signal for the event discarding block represents every DUT event that was not suppressed by the prescaler by a short positive pulse.
  - The width of this pulse depends on the primary timing
    - Its range is between 5.5 ns and 6.75 ns.

References and Additional Information:
Concept 129473
Concept 125212 – equations and rules of period and pulse width
Topic 144892 PRBS Jitter Measurement by TMU:
Prescaler is a frequency divider up to 1/16, making the input events less than 250 Msps this is the maximum event speed. For instance, 4 GHz by 16 makes 250 MHz. The prescaled events are decimated in the next stage to less than 50 Msps

Prescaler

- With very high data rates, it is possible that the distance between two events as selected by the prescaler is still smaller than the width of the pulse representing the first event. In this case, the second event will not be passed on to the event discarding block, which will invalidate the measurement.

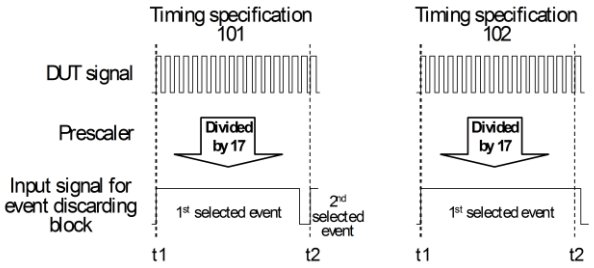


  - With specification 101, the distance between two selected events (at t1 and t2) is greater than the pulse width of the input signal for the event discarding block, but with specification 102,it is too small, and therefore the second selected event (at t2) will not be passed on to the event discarding block.
  - To summarize, the event discarding block requires that the distance between two events as selected by the prescaler is at least greater than 5.5 ns; and you can be sure that the described effect will not occur only if this distance is greater than 6.75 ns.

References and Additional Information:
Concept 129473

Prescaler

- Possible measures for high data rates
  Increase the prescaler value if not set to its maximum value of 16 (17 for event types RISE and FALL)
  If the prescaler is already set to 17 (as in the figure), the only option is to reduce the data rate.



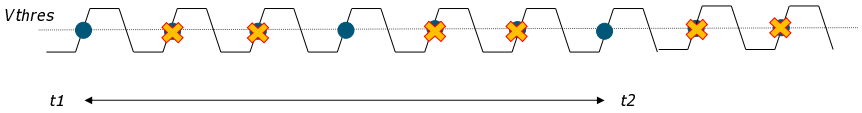
References and Additional Information:
Concept 129473

InterSkip - SmartRDI

- User-defined value to limit number of samples to be captured
- Specifies the number of consecutive samples to capture and the number of consecutive samples to discard
- If SMARTRDI is used then, if data rate is specified, the value will only take effect if it is greater than the value calculated from rdi.tmu().dataRate().
- Set to zero to ignore and capture every edge
- Works in conjunction with preScaler to condition signals higher than the 50 Msps spec

```
RDI_BEGIN();

    rdi.tmu("1").pin("PVI8_PMU_1") // PVI8_TMU
       .measMode(TA::RISE)
       .interSkip(0)
       .vth(1.0 V, 2.0 V)
       .samples(50)
       .execute();

    rdi.wait(100 us);

    rdi.tmu("2").pin("PS1600_1") // Digital TMU
       .measMode(TA::RISE)
       .interSkip(2)
       .vth(1.0 V)
       .samples(50)
       .execute();

    rdi.wait(100 us);

RDI_END();
```

interSkip = 2



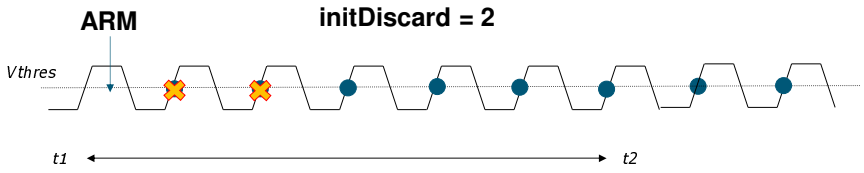
SmartRDI::

Picture is showing what would happen with prescaler set to 1

References and Additional Information:
Reference 149115

InitDiscard

- Sets the number of events to ignore before the samples are taken for TMU pin(s)
- Events to be captured start from the arm'ed vector
- Allows events to be ignored until desired time



References and Additional Information:
Reference 149114

DataRate - SmartRDI

- Sets the expected data rate for TMU pins
- SmartRdi statement: rdi.tmu().dataRate()
- Cannot be used with rdi.tmu().preScaler()
  - Mutually exclusive; Use one or the other.
  - DataRate will calculate the preScaler value
    - i.e. setting DataRate to 50Msps will force prescaler value to 2

SmartRDI::

References and Additional Information:
Reference 149111

Voltage Threshold

- By default, the TMU looks up the primary level setup to determine the event threshold. If one of the vth or voh resources is specified there, this voltage will be the threshold for the pins on which the TMU is run; the vol resource will only be used if none of the other two resources is present. But you can also specify a different voltage threshold.
- But note that if you define your own voltage threshold for a receive pin and execute a functional test for this pin in parallel with a TMU measurement, the functional test will also be executed with the modified voltage level. The original voltage levels will be restored by the system after the measurements.
- If a Rise/Fall time measurement is to be made, then two tests will be required with the voltage threshold set to the lower threshold for the first test and to the upper threshold for the second test for rising edges and opposite for falling edges.

References and Additional Information:
Concept 125212

Samples

- Sets the number of samples for each measurement of TMU pin(s)
- Can be used only in the pattern-based mode.

```
RDI_BEGIN();
    rdi.tmu().pin("PVI8_PMU_1")
             .measMode(TA::RISE_TIME)
             .vth(1.0 V,2.0 V)
             .samples(50)
             .execute();
    rdi.wait(100 us);
    rdi.tmu().pin("PS1600_1")
             .measMode(TA::RISE)
             .vth(1.0 V)
             .samples(50)
             .execute();
    rdi.wait(100 us);
RDI_END();
```


Example

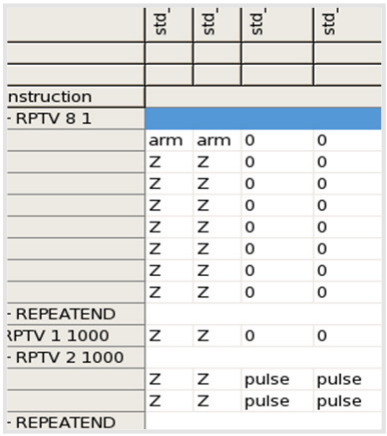
```
RDI_BEGIN();

  rdi.tmu().pin("PVI8_PMU_1")
        .measMode(TA::RISE_TIME)
        .vth(1.0 V,2.0 V)
        .samples(50)
        .execute();

  rdi.wait(100 us);

  rdi.tmu().pin("PS1600_1")
        .measMode(TA::RISE)
        .vth(1.0 V)
        .samples(50)
        .execute();

  rdi.wait(100 us);
```

RDI_END();

References and Additional Information:
Reference 149110

insertVec - SmartRDI

- Creates TMU events in specified pattern
- Specify the offset from label where you want to insert the TMU event
- In example “arm” denotes TMU event will be referenced to where arm is defined in wavetable



	std'	std'	std'	std'
nstruction				
- RPTV 8 1				
	arm	arm	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
- REPEATEND				
RPTV 1 1000	Z	Z	0	0
- RPTV 2 1000				
	Z	Z	pulse	pulse
	Z	Z	pulse	pulse
- REPEATEND				

SmartRDI::

Note The correct sequence of rdi.tmu().label() and .insertVec() must be strictly followed within the rdi ... execute() block if you want to insert TMU events into an already existing pattern.

References and Additional Information:

Reference 149121

insertVec - SmartRDI

TMU Setup - Timing

TMU setup specifics:

- Use r7 to generate arming signal (Recommendation)
- No compare on r7, r8 and no window compare for functional P/F testing when TMU is used
- Specify the exact time into the device cycle when the TMU is to be armed.

```
WAVETBL "tmu_wavetable"
PINS std_o
0 "d1:F00" 0
1 "d1:F10" 1
2 "d1:Z r7:ARM" arm
3 "d1:Z d2:Z" Z
brk ""
```

```
EQNSET 1 "tmu_example_eqn"
...
TIMINGSET 1 "tmu_example_tim"

PINS std_o
d1   = ...
d2   = ...
r7   = 2
```


SmartRDI::

Timing requirements for arming.

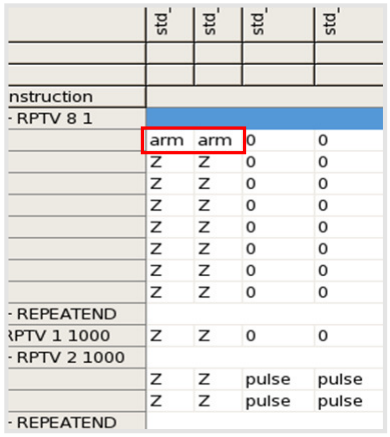
References and Additional Information:

insertVec - SmartRDI

TMU Setup - Pattern

TMU setup specifics:

- Place TMU arming waveforms in a pattern to specify exactly where to start a measurement
- Together with the arming edge position inside the device cycle, this determines the time reference t0 for the respective measurement



	std	std	std	std
nstruction				
- RPTV 8 1				
	arm	arm	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
	Z	Z	0	0
- REPEATEND				
RPTV 1 1000	Z	Z	0	0
- RPTV 2 1000				
	Z	Z	pulse	pulse
	Z	Z	pulse	pulse
- REPEATEND				

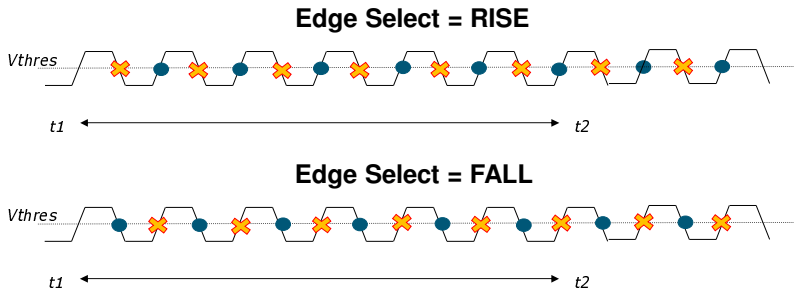
SmartRDI::

Multiple insertVec() calls can be made to insert multiple arms in the pattern. The number of measurements will be incremented by the number of arms inserted. Samples will be collected for each arm. If samples=10 and there are 2 insertVecs then data collected will be 2*10=20 samples total.

References and Additional Information:

Edge Select

- The edge type to trigger the event. It can be
  - RISE: rising edge
  - FALL: falling edge
  - RISE_FALL: alternate rising and falling edges.
    - This option is used for pulse and jitter measurements. In this case, the prescaler must be set to a value > 1 and < 17. Moreover, if an event discard is specified, it must be even (including 0).



References and Additional Information:
Reference 132458

TMU Test Types

- Frequency
- Period
- Pulse width
- Duty Cycle
- Pin to pin delay
- Rise/Fall Time

Tests and how to use the parameters (knobs)

TMU Test Type Coding Examples

- Will describe and show example code for each of test types
- Examples will include
  - SmartRDI – Test methods showing basic coding of test
  - MAPI - Test methods showing basic coding of test are included in the appendix

TMU Frequency Measurement

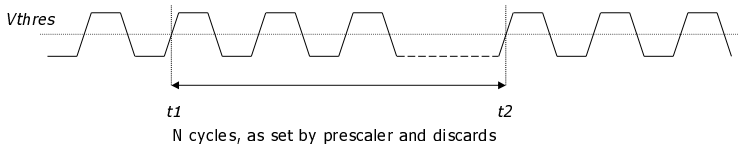
ADVANTEST

TMU Measurements - Frequency

PS1600
PS9G

Measurement principle:

- Take two time stamps t1, t2 of the same type of event (e.g., both rising slope) which are N clock cycles apart from each other.
- Frequency = N/(t2-t1)
- The bigger N, the more accurate the measurement (averaging)

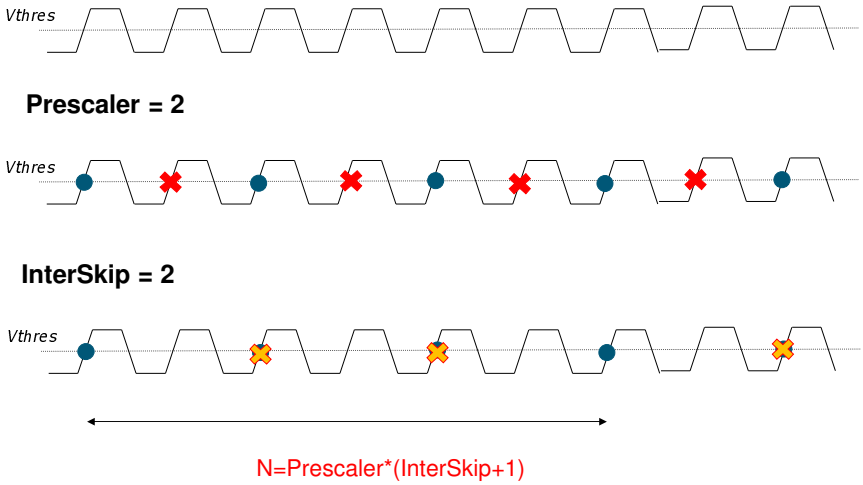


N cycles, as set by prescaler and discards

To measure frequency, it measures two same type of events t1 and t2.
Example shows setting rising slope as event.
After measured two events, upload data to DSP and calculate the frequency.
Setting larger N takes long but it gets more accurate measurement result.
N programs pre-scalar multiplies NB (number of events) plus one.

TMU Frequency Measurements – Prescaler & interSkip

PS1600
PS9G



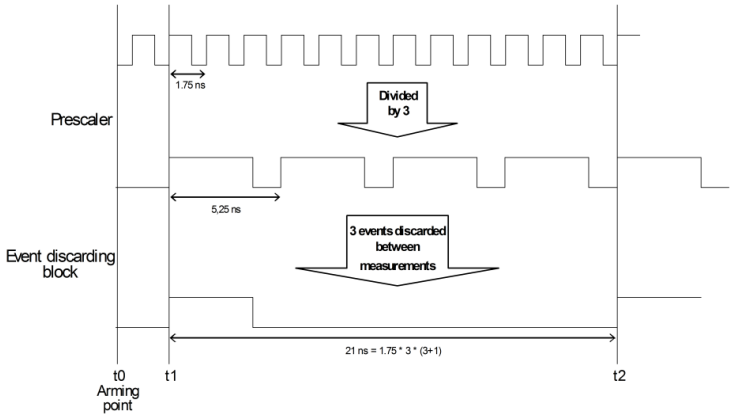
To measure frequency, it measures two same type of events t1 and t2.
Example shows setting rising slope as event.
After measured two events, upload data to DSP and calculate the frequency.
Setting larger N takes long but it gets more accurate measurement result.
N programs pre-scalar multiplies NB (number of events) plus one.

InterSampleDiscard from MAPI and interSkip are the same.

TMU Frequency Measurements – Prescaler & InterSkip

Obtaining Period for frequency

- **Signal to measure must meet**
  - **Sampling rate = 20nS**
  - **Discarding block minimum distance**
- Example shows 1.75nS period expected
- Rising edge captured
- Prescaler = 3 to do divide by 3
- InterSkip = 3 to discard 3 events between samples to satisfy the sample rate



Period = (t2 – t1) / N where N=Prescaler*(InterSkip+1)

To measure frequency, it measures two same type of events t1 and t2.
Period = (t2-t1) / N where N = (prescaler * (interskip + 1))
Example shows setting rising slope as event.
After measured two events, upload data to DSP and calculate the frequency.
Setting larger N takes long but it gets more accurate measurement result.
N programs pre-scalar multiplies NB (number of events) plus one.

Topic 125212:
Notes on period measurement

- By the combined effects of prescaler and event discarding block, eleven events (rising edges) are skipped between the first sample (at t1) and the second sample (at t2).
- Since the prescaler is set to a value > 1, a certain number of events before the first sample is ignored. The graphic assumes that only one event is skipped. But if the signal to be measured is already present when the TMU is armed, the number of skipped events can vary.
- The same effect could be achieved in this example by setting the prescaling factor to 12 and the events to be discarded between samples to 0.
- A typical application of event discarding between samples would be a period measurement consisting of only two time stamps which are taken at a very large

distance. In this case increasing the prescaler factor alone will not be sufficient.

RDI: TMU Programming – Frequency Measurement

```
RDI_INIT();
ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(m_sSuite);
        cerr << "Test Suite: " << m_sSuite << endl;
        CONNECT(); // For RISE/FALL measurements without DIB

                RDI_BEGIN(m_rdiMode);
                        rdi.port(m_sPortLabel).tmu(1)
                                        .label(m_sPattLabel)
                                        .pin(m_sStartPins)
                                        .measNum(m_iNumMeasurementArms)
                                        .insertVec(m_iVecOffset[1])
                                        .insertVec(m_iVecOffset[2])
                                        .measMode(m_eStartEdgeTypeRDI)
                                        .dataRate(m_dDataRate)
                                        .vth(m_dThresholdA)
                                        .samples(m_iNumSamples)
                                        .initDiscard(m_iInitialDiscard)
                                        .cont();

                        rdi.port(m_sPortLabel).tmu(1).execute();
                        rdi.port(m_sPortLabel).wait(5e-3);
                RDI_END();

        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(curr_site);
if(tmu_ok) {
        LogResults(m_sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << m_sSuite << "\", SITE[" << curr_site << "] *** !!!FAILED
}
return;
```

SMARTRDI TMU Setup Using DataRate

Define Pattern Label and Pin(s) to work with.

Define TMU Task:
- Select where in pattern to place ARM (.insertVec)
- Specify number of ARM measurements (.measNum)
- Select the edge type (.measMode)
- **Set expected data rate (.dataRate)**
- Set voltage threshold (.vth)
- Set Samples (.samples)
- Specify initial events to discard (.initDiscard)

Retrieve the measurement results

Calculate frequency

RDI:: Code using Datarate specified
This slide shows an excerpt of the SmartRDI code in the example program. Just what is needed to perform a TMU measurement is shown. The example program had sections for doing StartPins and StopPins. It also had code to perform 2 different measurements (m_iNumShots). To fit within the confines of a slide this has been reduced to show the bare minimum. The slides demonstrates the ON_FIRST_INVOCATION BEGIN and END block to setup TMU and perform the measurement. This is followed by result processing section where the TMU measured time stamps are retrieved in getResultCache() and processed in LogResults() to calculate the Frequency result.

The code shown is the basic building block for setting up RDI TMU measurements. All of the test types that do a single shot measurement will be set up similar to this. The only real difference is the calculation of the result and this is selected by a test method parameter.

A two shot measurement would be a RISE or FALL time measurement since multiple voltage thresholds are needed. Then there would be two rdi statements with a different voltage threshold .vth() specified for each. The LogResults() function has to use the two sets of data to calculate the RISE or FALL time result.

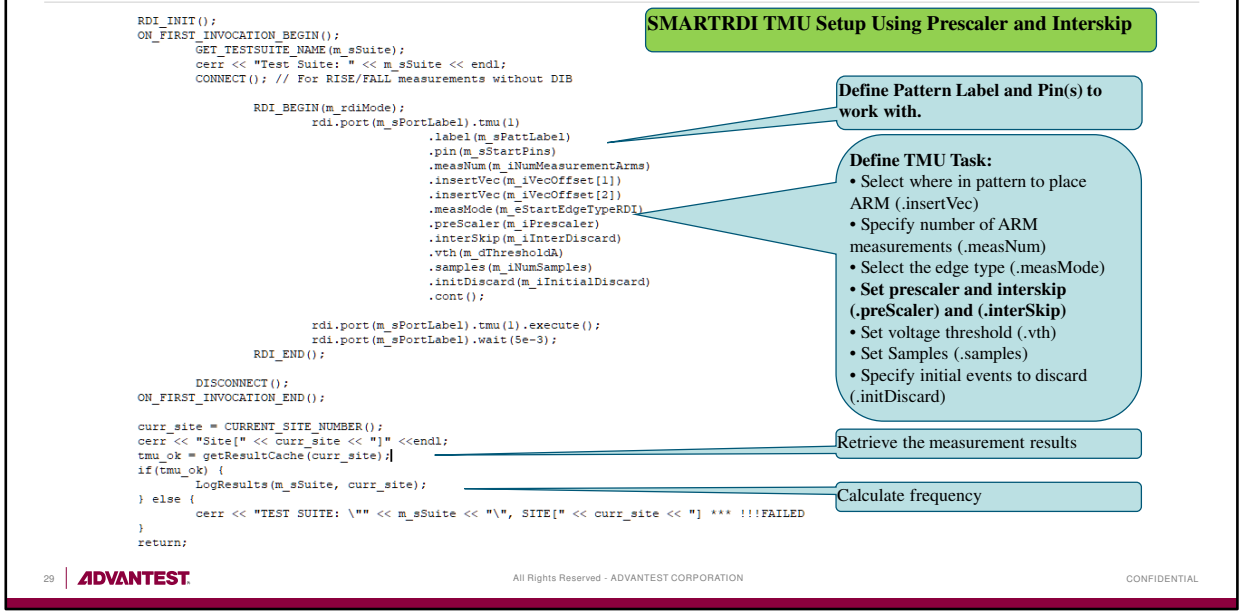
RDI: TMU Programming – Frequency Measurement

```
RDI_INIT();
ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(m_sSuite);
        cerr << "Test Suite: " << m_sSuite << endl;
        CONNECT(); // For RISE/FALL measurements without DIB

                RDI_BEGIN(m_rdiMode);
                        rdi.port(m_sPortLabel).tmu(1)
                                        .label(m_sPattLabel)
                                        .pin(m_sStartPins)
                                        .measNum(m_iNumMeasurementArms)
                                        .insertVec(m_iVecOffset[1])
                                        .insertVec(m_iVecOffset[2])
                                        .measMode(m_eStartEdgeTypeRDI)
                                        .dataRate(m_dDataRate)
                                        .vth(m_dThresholdA)
                                        .samples(m_iNumSamples)
                                        .initDiscard(m_iInitialDiscard)
                                        .cont();

                        rdi.port(m_sPortLabel).tmu(1).execute();
                        rdi.port(m_sPortLabel).wait(5e-3);
                RDI_END();

        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(curr_site);
if(tmu_ok) {
        LogResults(m_sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << m_sSuite << "\", SITE[" << curr_site << "] **
}
return;
```

Test Method	tmu_ppt_tml.t_tmu_rdi_template.tmu_rdi_template
Parameters	...
Exec	
Setup	
Application Type	FREQUENCY
Number Of Meast	1
Samples Per Meas	2
DataRate	0[MHz]
Prescaler	1
Inter Sample Disc	9
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST
Start Pin Edge Typ	RISE
Stop Pin	
Stop Pin Edge Typ	NONE
RDI	
Mode	BURST
Pattern Label	tmu_rdi_test_patt
Port Label	tmu_port
Vector Offset1	4
Vector Offset2	4
Limits	...

RDI::
This slide shows an excerpt of the SmartRDI code in the example program. The test method parameters are shown to correlate where the RDI inputs are coming from.
Test Method Parameters-
Application Type: FREQUENCY – This sets what calculation type will be performed on the measurements.

RDI: TMU Programming – Frequency Measurement




```
RDI_INIT();
ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(m_sSuite);
        cerr << "Test Suite: " << m_sSuite << endl;
        CONNECT(); // For RISE/FALL measurements without DIB

                RDI_BEGIN(m_rdiMode);
                        rdi.port(m_sPortLabel).tmu(1)
                                        .label(m_sPattLabel)
                                        .pin(m_sStartPins)
                                        .measNum(m_iNumMeasurementArms)
                                        .insertVec(m_iVecOffset[1])
                                        .insertVec(m_iVecOffset[2])
                                        .measMode(m_eStartEdgeTypeRDI)
                                        .preScaler(m_iPrescaler)
                                        .interSkip(m_iInterDiscard)
                                        .vth(m_dThresholdA)
                                        .samples(m_iNumSamples)
                                        .initDiscard(m_iInitialDiscard)
                                        .cont();

                        rdi.port(m_sPortLabel).tmu(1).execute();
                        rdi.port(m_sPortLabel).wait(5e-3);
                RDI_END();

        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(curr_site);
if(tmu_ok) {
        LogResults(m_sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << m_sSuite << "\", SITE[" << curr_site << "] *** !!!FAILED
}
return;
```


RDI:: Code using Prescaler/Interskip specified

This slide shows an excerpt of the SmartRDI code in the example program. To fit within the confines of a slide this has been reduced to show the bare minimum. The slides demonstrates the ON_FIRST_INVOCATION BEGIN and END block to setup TMU using the **preScaler and Interskip** and perform the measurement. This is followed by result processing section where the TMU measured time stamps are retrieved in getResultCache() and processed in LogResults() to calculate the Frequency result.

TMU Programming – Frequency Measurement

- Calculation of the Frequency occurs in the LogResults() function of the example code. See Appendix slide: c_tmu_base.cpp – LogResults() for complete code example.
- Function checks the selection from the parameters and calls correct calculation as shown in code snippet below.

```
if(m_sAppType == "PERIOD") {
    CalcPeriod(sname, site);
} else if(m_sAppType == "FREQUENCY") {
    CalcFrequency(sname, site);
} else if((m_sAppType == "DUTYCYCLE") || (m_sAppType == "PW")){
    CalcDutyCycle(sname, site);
} else if(m_sAppType == "P2P") {
    CalcPin2PinDelay(sname, site);
} else if((m_sAppType == "RISE") || (m_sAppType == "FALL")){
    CalcRiseFall(sname, site);
} else if(m_sAppType == "JITTER") {
    CalcJitter(sname, site);
} else if(m_sAppType == "EYE") {
    CalcEye(sname, site);
} else if(m_sAppType == "DRIFT") {
    CalcDrift(sname, site);
} else {
    CalcNone(sname, site);
}
```

See Appendix Slide: c_tmu_base.cpp – LogResults()

Test Method	tmu_ppt_tml.t_tmu_rdi_template.tmu_rdi_template
Parameters	...
Exec	
Setup	
Application Type	FREQUENCY
Number Of Meas	1
Samples Per Mea	2
DataRate	0[MHz]
Prescaler	1
Inter Sample Disc	9
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST
Start Pin Edge Typ	RISE
Stop Pin	
Stop Pin Edge Typ	NONE
RDI	
Mode	BURST
Pattern Label	tmu_rdi_test_patt
Port Label	tmu_port
Vector Offset1	4
Vector Offset2	4
Limits	...

This slide shows an excerpt of the result calculation code in the example program. The LogResults selects which calculations to perform on the time stamps. Application Type: FREQUENCY – This sets what calculation type will be performed on the measurements.

LogResults method:

```
void tmu_base::LogResults(string sname, int site)
{
        int pin_idx, shot_idx, num_pins;

        map<string,ARRAY_D>::iterator st, sp;

        for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                num_pins = m_vecStartPins.size();
                for(pin_idx=0; pin_idx<num_pins; pin_idx++) {
                        st =
m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(st !=
m_mapStartEvents[shot_idx].end()) {
                                cerr << "\tShot
```

```
Number[" << shot_idx << "]: " << "Start Pin <" << st->first << "> Data = " << st-
>second <<  endl;
                                                }
                                }

                                num_pins = m_vecStopPins.size();
                                for(pin_idx=0; pin_idx<num_pins; pin_idx++) {
                                                sp =
m_mapStopEvents[shot_idx].find(m_vecStopPins[pin_idx]);
                                                if(sp !=
m_mapStopEvents[shot_idx].end()) {
                                                                cerr << "\tShot
Number[" << shot_idx << "]: " << "Stop  Pin <" << sp->first << "> Data = " << sp-
>second <<  endl;
                                                }
                                }
                }

                if(m_sAppType == "PERIOD") {
                                CalcPeriod(sname, site);
                } else if(m_sAppType == "FREQUENCY") {
                                CalcFrequency(sname, site);
                } else if((m_sAppType == "DUTYCYCLE") || (m_sAppType == "PW")){
                                CalcDutyCycle(sname, site);
                } else if(m_sAppType == "P2P") {
                                CalcPin2PinDelay(sname, site);
                } else if((m_sAppType == "RISE") || (m_sAppType == "FALL")){
                                CalcRiseFall(sname, site);
                } else if(m_sAppType == "JITTER") {
                                CalcJitter(sname, site);
                } else if(m_sAppType == "EYE") {
                                CalcEye(sname, site);
                } else if(m_sAppType == "DRIFT") {
                                CalcDrift(sname, site);
                } else {
                                CalcNone(sname, site);
                }

                ON_LAST_INVOCATION_BEGIN();
                                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++)
{
                                                m_mapStartEvents[shot_idx].clear();
                                                m_mapStopEvents[shot_idx].clear();
                                }
```

```
                                cerr << endl;
                ON_LAST_INVOCATION_END();
}
```

TMU Programming – Frequency Measurement

- CalcFrequency() takes time stamps and performs calculations
- Calculates frequency for each start pin specified.
- Number of shots should be 1 for frequency measurement
- PostScaleResult() handles adjusting the value based upon preScaler and interSkip values

```
void tmu_base::CalcFrequency(string sname, int site)
{
        size_t pin_idx;
        int arm_idx, shot_idx, smp_idx;
        double result;
        ARRAY_D events[3];

        map<string,ARRAY_D>::iterator it;
        cerr << endl << "Calculating \"FREQUENCY\" Results for Site[" << site << "]" << endl << endl;

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(it != m_mapStartEvents[shot_idx].end()) {
                                events[shot_idx] = it->second;
                                result = 0.0;
                                for(arm_idx=0; arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                        ARRAY_D arms(m_iNumSamples);
                                        for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {
                                                arms[smp_idx] = events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                        }
                                        Wave<double> wav(arms);
                                        int len = wav.getSize();
                                        double val;
                                        val = wav.differentiate().select(1,1, len-1).mean();
                                        val = PostScaleResult(val);
                                        result += val;
                                }
                                result /= (double)m_iNumMeasurementArms;
                                JudgeAndLog(m_vecStartPins[pin_idx], "freq", 1.0/result, site);
                        } else {

                        }
                }
        }
}
```


This slide shows an excerpt of the result calculation code in the example program. The CalcFrequency function expects an edge mode of RISE or FALL. It also expects a number of shots parameter of 1 since only one TMU setup and execution is needed (Code does have a for loop for number of shots but should only go through this loop once). It calculates average of the distance between time stamps. This value is then adjusted based upon the preScaler and interSkip values from the TMU setup in the PostScaleResult(). Finally if multiple ARMS had been placed in pattern then the result average across each ARM samples is calculated. At this point we have the period result. Finally the frequency is calculated from the period.

TMU Programming – Frequency Measurement

- PostScaleResult() handles adjusting the value based upon preScaler and interSkip values
- If DataRate is specified then preScaler and interSkip (interdiscard) are read back
- If DataRate is not specified then preScaler and interSkip (interdiscard) entered in parameters are used
- Calculation

  result = val / (prescaler * (interskip+1);

```
double tmu_base::PostScaleResult(double val)
{
    double result;
    double prescale;
    double interskip;

    if(m_bUseDataRate) {
            prescale = (double)m_drSettings.prescaler;
            interskip = (double)m_drSettings.interdiscard;
    } else {
            prescale = (double)m_iPrescaler;
            interskip = (double)m_iInterDiscard;
    }
    cout << "prescale=" << prescale << endl;
    cout << "interskip=" << interskip << endl;
    result = val/(prescale*(interskip+1.0));

    return result;
}
```


This slide shows an excerpt of the result calculation code in the example program.

TMU Period Measurement

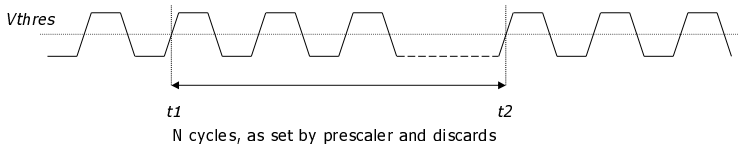
ADVANTEST

MAPI: TMU Measurements - Period

PS1600
PS9G

Measurement principle:

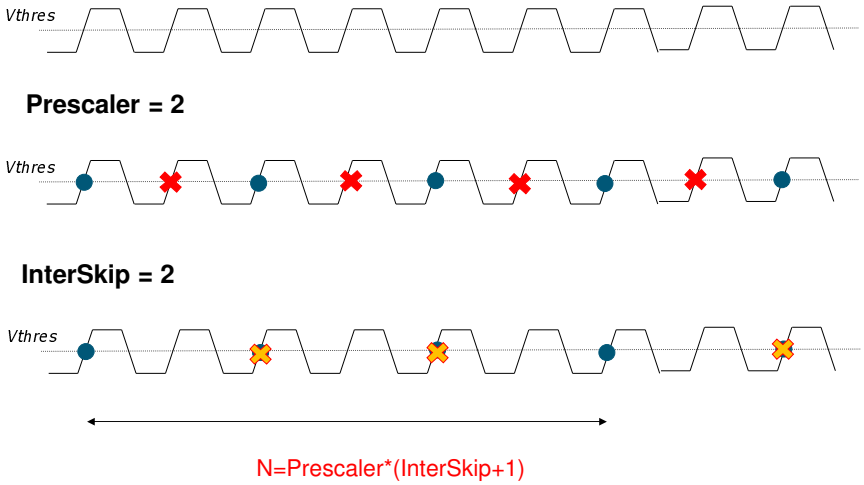
- Take two time stamps t1, t2 of the same type of event (e.g., both rising slope) which are N clock cycles apart from each other.
- Period = (t2-t1)/N
- The bigger N, the more accurate the measurement (averaging)



N cycles, as set by prescaler and discards

TMU Period Measurements – Prescaler And InterSkip

PS1600
PS9G



To measure period, it measures two same type of events t1 and t2.
Example shows setting rising slope as event.
After measured two events, upload data to DSP and calculate the period.
Setting larger N takes long but it gets more accurate measurement result.
N programs pre-scalar multiplies NB (number of events) plus one.

RDI: TMU Programming – Period Measurement

```
RDI_INIT();
ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(m_sSuite);
        cerr << "Test Suite: " << m_sSuite << endl;
        CONNECT(); // For RISE/FALL measurements without DIB

                RDI_BEGIN(m_rdiMode);
                        rdi.port(m_sPortLabel).tmu(1)
                                        .label(m_sPattLabel)
                                        .pin(m_sStartPins)
                                        .measNum(m_iNumMeasurementArms)
                                        .insertVec(m_iVecOffset[1])
                                        .insertVec(m_iVecOffset[2])
                                        .measMode(m_eStartEdgeTypeRDI)
                                        .dataRate(m_dDataRate)
                                        .vth(m_dThresholdA)
                                        .samples(m_iNumSamples)
                                        .initDiscard(m_iInitialDiscard)
                                        .cont();

                        rdi.port(m_sPortLabel).tmu(1).execute();
                        rdi.port(m_sPortLabel).wait(5e-3);
                RDI_END();

        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(curr_site);
if(tmu_ok) {
        LogResults(m_sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << m_sSuite << "\", SITE[" << curr_site << "] ***
}
return;
```

Test Method	tmu_ppt_tml.t_tmu_rdi_template.tmu_rdi_template
Parameters	...
Exec	
Setup	
Application Type	PERIOD
Number Of Measu	1
Samples Per Meas	2
DataRate	20[MHz]
Prescaler	1
Inter Sample Disc	9
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST,TMU_SP
Start Pin Edge Typ	RISE
Stop Pin	
Stop Pin Edge Typ	FALL
RDI	
Mode	BURST
Pattern Label	tmu_rdi_test_patt
Port Label	tmu_port
Vector Offset1	4
Vector Offset2	4
Limits	...

See Appendix Slide: c_tmu_base.cpp – LogResults()

RDI::
An example setup to do Period measurement using DataRate. Code is same as for Frequency measurement with the exception that the Application Type is set to PERIOD for the LogResult() calculation.
The test method parameters are shown to correlate where the RDI inputs are coming from.
Test Method Parameters-
Application Type: Period – This sets what calculation type will be performed on the measurements.

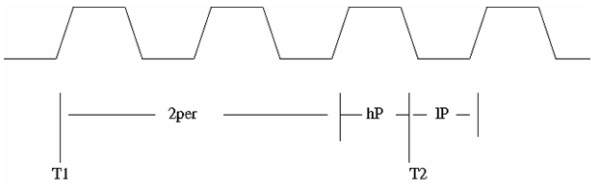
See Appendix Slide: c_tmu_base.cpp – LogResults()

TMU Pulse Width Measurement

ADVANTEST

Pulse Width Measurement

- Requirement is to measure the pulse width of signals with TMU.
- Both high pulse width and low pulse width are measured. The time stamps of some rising edges and falling edges are captured.
- The high pulse width (hp) and low pulse width (lp) are calculated from the timing stamps. The number of events discarded between two samples must be even. Otherwise the same edge is always captured.



References and Additional Information:
Reference 126959

Pulse Width Measurement

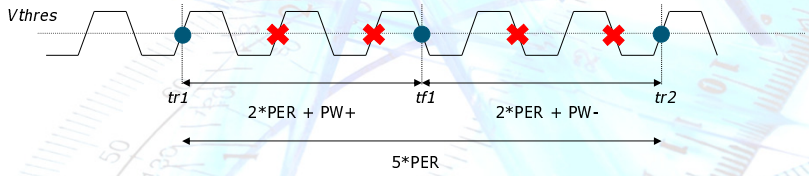
PS1600
PS9G

Measurement principle:

- Take time stamps of two rising edges tr1, tr2 and one falling edge tf1. Using the equations in figure below you can extract PW+ and PW-.

TMU setup specifics:

- Minimum per-scaler setting of alternating edges (e.g., rise→fall) is 2
- Pre-scaler will drop 2 same-edge events and then lets the alternate edge pass through
- The number of events to be discarded between two samples must be even.



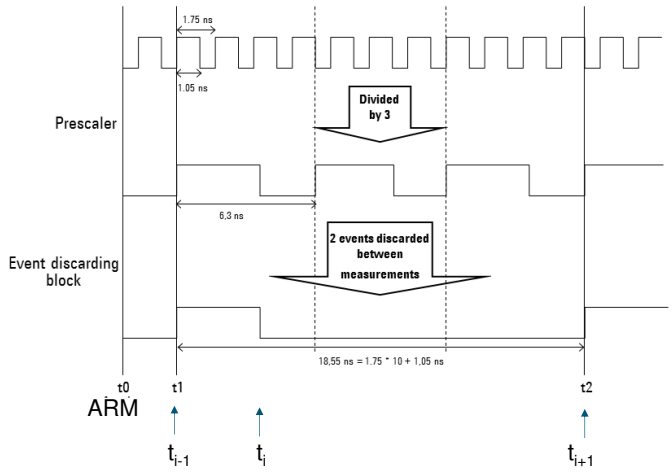
To measure a pulse width, it measures two rising edges tr1 and tr2 and one falling edges tf1.
You then calculate PW+ and PW- by using equations in figure below.
It is important on the TMU setup that
Minimum pre-scalar setting of alternating edges like rise to fall is 2.
And pre-scalar drops two same-edge events and lets the alternate edge pass through.

TMU Pulse Width Measurements – Prescaler & InterSkip

Obtaining Pulse Width

- **Signal to measure must meet**
  - **Sampling rate = 20nS**
  - **Discarding block minimum distance**

- Example shows 1.75nS period expected
- Rising and Falling edge captured
- Prescaler = 3 to do divide by 3
- InterSkip = 2 to discard 2 events between samples



PW+ = $(t_i - t_{i-1})$ MOD Period PW- = $(t_{i+1}-t_i)$ MOD Period

Need edge mode of RISE_FALL
Explanation of TDC content compared to program equations:
Program example code equations where multiples of 3 samples are collected and the index i starts at 0:

```
period = (arms[i+2]-arms[i]) / ((prescale+0.5)*(interskip+1)*2);
thi = fmod(arms[i+1]-arms[i],period);
tlo = fmod(arms[i+2]-arms[i+1],period);
```

Compared to equations in slide where i appears to start at 1.

Topic 125212:
Note the following points with respect to alternate sampling of rising and falling edges:

- The event discarding block only considers the rising edges of the prescaler output; the falling edges are ignored. Therefore all selected events (that is, also the falling edges) are represented by the prescaler as rising edges.
- If in this case the prescaler is set to n, then after selecting an edge, it skips the next n edges of the same type, and then selects the next edge of the opposite type. Thus in the example, after selecting a rising edge, it skips the next 3 rising edges and then selects the next falling edge; and after selecting a falling edge, it skips the next 3 falling edges, and then selects the next rising edge.

- If an even number (including 0) of events is to be discarded before the first sample, the first sampled event will be a rising edge. If you discard an odd number of events before the first sample, the first sampled event will be a falling edge. In this case the formulas for PW+ and PW-, as specified in the graphic, must be interchanged.
- If you specify an odd number of events to be discarded between samples (for example, only one event in the figure above), the sampled events will all be of the same type, that is, they will be either all rising or all falling edges. Therefore, if you want to measure the pulse width, the number of events to be discarded between two samples must be even.
- With alternate sampling of rising and falling edges, the period length can be calculated from the samples by the formula

This is in Help: $(t_{i+1} - t_i)$ / ((prescaling_factor + 0.5) * (inter_meas_discard + 1)).

Think it needs to be: $(t_{i+1} – t_{i-1})$ / ((prescaling_factor + 0.5) * (inter_meas_discard + 1)).

RDI: TMU Programming – Pulse Width Measurement

```
RDI_INIT();
ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(m_sSuite);
        cerr << "Test Suite: " << m_sSuite << endl;
        CONNECT(); // For RISE/FALL measurements without DIB

                RDI_BEGIN(m_rdiMode);
                        rdi.port(m_sPortLabel).tmu(1)
                                        .label(m_sPattLabel)
                                        .pin(m_sStartPins)
                                        .measNum(m_iNumMeasurementArms)
                                        .insertVec(m_iVecOffset[1])
                                        .insertVec(m_iVecOffset[2])
                                        .measMode(m_eStartEdgeTypeRDI)
                                        .preScaler(m_iPrescaler)
                                        .interSkip(m_iInterDiscard)
                                        .vth(m_dThresholdA)
                                        .samples(m_iNumSamples)
                                        .initDiscard(m_iInitialDiscard)
                                        .cont();

                                rdi.port(m_sPortLabel).tmu(1).execute();
                                rdi.port(m_sPortLabel).wait(5e-3);
                        RDI_END();

        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(curr_site);
if(tmu_ok) {
        LogResults(m_sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << m_sSuite << "\", SITE[" << curr_site << "] ***
}
return;
```

Test Method	tmu_ppt_tml.t_tmu_rdi_template.tmu_rdi_template
Parameters	...
Exec	
Setup	
Application Type	PW
Number Of Measu	1
Samples Per Meas	3
DataRate	0[MHz]
Prescaler	2
Inter Sample Disc	0
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST
Start Pin Edge Typ	RISE_FALL
Stop Pin	
Stop Pin Edge Typ	NONE
RDI	
Mode	BURST
Pattern Label	tmu_rdi_test_patt
Port Label	tmu_port
Vector Offset1	4
Vector Offset2	4
Limits	...

See Appendix Slide: c_tmu_base.cpp – LogResults()

RDI::
An example setup to do Pulse Width measurement using Prescaler and interskip.
Code is same as for Frequency measurement with the exception that the Application Type is set to PW for the LogResult() calculation.
See Appendix Slide: c_tmu_base.cpp – LogResults()

The test method parameters are shown to correlate where the RDI inputs are coming from.
Test Method Parameters-
Application Type: PW – This sets what calculation type will be performed on the measurements.
Start Pin Edge Type: RISE_FALL – Required to calculate the high duration of the pulse and the low duration.
Number of Shots: 1 – Pulse Widthonly requires one TMU execution

TMU Programming – Pulse Width Measurement

- **Pulse Width and Duty Cycle use same calculation function.**
- CalcDutyCycle () takes time stamps and performs calculations
- Calculates high pulse for each start pin specified.
- Number of shots should be 1 for Pulse Width measurement
- Edge Mode expected is RISE_FALL
- Parameters should have start pin specified

```
void tmu_base::CalcDutyCycle(string sname, int site)
{
        size_t pin_idx;
        int arm_idx, shot_idx, smp_idx;
        double result;
        double resPeriod;
        double resThi;
        double resTlo;
        ARRAY_D events[3];
        double prescale = 0.0;
        double interskip = 0.0;

        map<string,ARRAY_D>::iterator it;
        if(m_sAppType == "DUTYCYCLE") {
                cerr << endl << "Calculating \"DUTY CYCLE\" Results for Site[" << site << "]" << endl << endl;
        }
        else {
                cerr << endl << "Calculating \"PulseWidth\" Results for Site[" << site << "]" << endl << endl;
        }

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(it != m_mapStartEvents[shot_idx].end()) {
                                events[shot_idx] = it->second;
                                result = 0.0;
                                for(arm_idx=0; arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                        ARRAY_D arms(m_iNumSamples);
                                        double period=0;
                                        double thi=0;
                                        double tlo=0;

                                        for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {
                                                arms[smp_idx] = events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                        }

                                        // array is setup !!!
                                        if(m_bUseDataRate) {
                                                prescale = (double)m_drSettings.prescaler;
                                                interskip = (double)m_drSettings.interdiscard;
                                        } else {
                                                prescale = (double)m_iPrescaler;
                                                interskip = (double)m_iInterDiscard;
                                        }
```


Pulse Width and Duty Cycle use same calculation function because calculations compute same values.

This slide shows an excerpt of the result calculation code in the example program. The CalcDutyCycle function expects an edge mode of RISE_FALL. It also expects a number of shots parameter of 1 since only one TMU setup and execution is needed (Code does have a for loop for number of shots but should only go through this loop once). It calculates high pulse from time stamps. Finally if multiple ARMS had been placed in pattern then the result average across each ARM samples is calculated. Finally the high pulse is returned as the pulse width.

TMU Duty Cycle Measurement

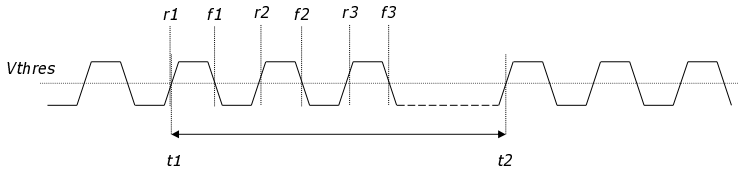
ADVANTEST

MAPI: TMU Measurements – Duty Cycle

PS1600
PS9G

Measurement principle:

- Capture RISE_FALL events
- Calculate Period = (t2-t1)/N
- Calculate High time = (sum of falling edge minus rising edge) / N
- The bigger N, the more accurate the measurement (averaging)
- Duty Cycle = High Time / Period * 100



N cycles, as set by prescaler and discards

RDI: TMU Programming – Duty Cycle Measurement

```
RDI_INIT();
ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(m_sSuite);
        cerr << "Test Suite: " << m_sSuite << endl;
        CONNECT(); // For RISE/FALL measurements without DIB

                RDI_BEGIN(m_rdiMode);
                        rdi.port(m_sPortLabel).tmu(1)
                                        .label(m_sPattLabel)
                                        .pin(m_sStartPins)
                                        .measNum(m_iNumMeasurementArms)
                                        .insertVec(m_iVecOffset[1])
                                        .insertVec(m_iVecOffset[2])
                                        .measMode(m_eStartEdgeTypeRDI)
                                        .preScaler(m_iPrescaler)
                                        .interSkip(m_iInterDiscard)
                                        .vth(m_dThresholdA)
                                        .samples(m_iNumSamples)
                                        .initDiscard(m_iInitialDiscard)
                                        .cont();

                                rdi.port(m_sPortLabel).tmu(1).execute();
                                rdi.port(m_sPortLabel).wait(5e-3);
                        RDI_END();

        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(curr_site);
if(tmu_ok) {
        LogResults(m_sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << m_sSuite << "\", SITE[" << curr_site << "] *** !
}
return;
```

Test Method	tmu_ppt_tml.t_tmu_rdi_template.tmu_rdi_template
Parameters	...
Exec	
Setup	
Application Type	DUTYCYCLE
Number Of Measu	1
Samples Per Mea	3
DataRate	0[MHz]
Prescaler	2
Inter Sample Disc	0
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST
Start Pin Edge Typ	RISE_FALL
Stop Pin	
Stop Pin Edge Typ	NONE
RDI	
Mode	BURST
Pattern Label	tmu_rdi_test_patt
Port Label	tmu_port
Vector Offset1	4
Vector Offset2	4
Limits	...

See Appendix Slide: c_tmu_base.cpp – LogResults()

RDI::
An example setup to do Duty Cycle measurement using Prescaler and interSkip.
Code is same as for Frequency measurement with the exception that the Application Type is set to DUTYCYCLE for the LogResult() calculation.
See Appendix Slide: c_tmu_base.cpp – LogResults()

The test method parameters are shown to correlate where the RDI inputs are coming from.
Test Method Parameters-
Application Type: DUTYCYCLE – This sets what calculation type will be performed on the measurements.
Start Pin Edge Type: RISE_FALL – Required to calculate the high duration of the pulse and the low duration.
Number of Shots: 1 – Duty Cycle only requires one TMU execution

TMU Programming – Duty Cycle Measurement

- **Pulse Width and Duty Cycle use same calculation function.**
- CalcDutyCycle() takes time stamps and performs calculations
- Calculates period and high pulse for each start pin specified.
- Number of shots should be 1 for Pulse Width measurement
- Edge Mode expected is RISE_FALL
- Parameters should have start pin specified

```
void tmu_base::CalcDutyCycle(string sname, int site)
{
        size_t pin_idx;
        int arm_idx, shot_idx, smp_idx;
        double result;
        double resPeriod;
        double resThi;
        double resTlo;
        ARRAY_D events[3];
        double prescale = 0.0;
        double interskip = 0.0;

        map<string,ARRAY_D>::iterator it;
        if(m_sAppType == "DUTYCYCLE") {
                cerr << endl << "Calculating \"DUTY CYCLE\" Results for Site[" << site << "]" << endl << endl;
        }
        else {
                cerr << endl << "Calculating \"PulseWidth\" Results for Site[" << site << "]" << endl << endl;
        }

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(it != m_mapStartEvents[shot_idx].end()) {
                                events[shot_idx] = it->second;
                                result = 0.0;
                                for(arm_idx=0; arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                        ARRAY_D arms(m_iNumSamples);
                                        double period=0;
                                        double thi=0;
                                        double tlo=0;

                                        for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {
                                                arms[smp_idx] = events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                        }

                                        // array is setup !!!
                                        if(m_bUseDataRate) {
                                                prescale = (double)m_drSettings.prescaler;
                                                interskip = (double)m_drSettings.interdiscard;
                                        } else {
                                                prescale = (double)m_iPrescaler;
                                                interskip = (double)m_iInterDiscard;
                                        }
}
```


Pulse Width and Duty Cycle use same calculation function because calculations compute same values.

This slide shows an excerpt of the result calculation code in the example program. The CalcDutyCycle function expects an edge mode of RISE_FALL. It also expects a number of shots parameter of 1 since only one TMU setup and execution is needed (Code does have a for loop for number of shots but should only go through this loop once). It calculates period and high pulse from time stamps. Finally if multiple ARMS had been placed in pattern then the result average across each ARM samples is calculated. Finally the duty cycle is calculated by dividing the high pulse time by the period and multiplied by 100.

TMU Pin to Pin Delay Measurement

ADVANTEST

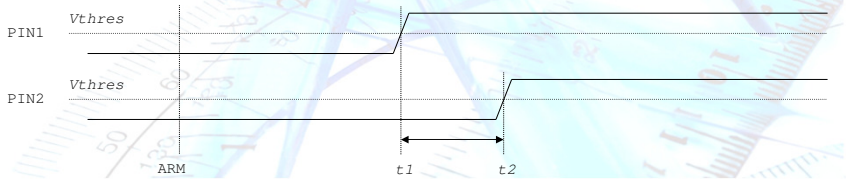
TMU Programming – Pin to Pin Delay Measurement

Measurement principle:

- Place arming at exactly same time on both pins via timing equations and arming cycle number in pattern
- Pin-to-pin skew is the difference between the measurements on the two pins: skew=t2-t1

TMU setup specifics:

- when PS is used, the first rising edge is always discarded; in case the very first event after arming needs to be measured, use PS=1.



To measure the pin to pin skew or delay, place arming at exactly same time on both pins.
It is set by timing equations and arming cycle number in test pattern file.
After measured both events, calculate time difference on both pins.
one falling edges tf1.
When pre-scalar is used, the first edge is always discarded.
In case, the very first event after arming needs to be measures, set pre-scalar value to 1 which ignores pre-scalar function.

RDI: TMU Programming – Pin to Pin Delay Measurement

```
RDI_INIT();
ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(m_sSuite);
        cerr << "Test Suite: " << m_sSuite << endl;
        CONNECT(); // For RISE/FALL measurements without DIB

                RDI_BEGIN(m_rdiMode);
                    if(m_bUseStopPin) {
                            rdi.port(m_sPortLabel).tmu(2)
                                    .label(m_sPattLabel)
                                    .pin(m_sStopPins)
                                    .measNum(m_iNumMeasurementArms)
                                    .insertVec(m_iVecOffset[1])
                                    .insertVec(m_iVecOffset[2])
                                    .measMode(m_eStopEdgeTypeRDI)
                                    .dataRate(m_dDataRate)
                                    .vth(m_dThresholdA)
                                    .samples(m_iNumSamples)
                                    .initDiscard(m_iInitialDiscard)
                                    .cont();
                    }
                        rdi.port(m_sPortLabel).tmu(1).execute();
                        rdi.port(m_sPortLabel).wait(5e-3);
                RDI_END();

        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(curr_site);
if(tmu_ok) {
        LogResults(m_sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << m_sSuite << "\", SITE[" << curr_site << "] **
}
return;
```

Stop Pin requires its own setup

See Appendix Slide: c_tmu_base.cpp – LogResults()

Test Method	tmu_ppt_tml.t_tmu_rdi_template.tmu_rdi_template
Parameters	...
Exec	
Setup	
Application Type	P2P
Number Of Measu	1
Samples Per Meas	10
DataRate	25[MHz]
Prescaler	1
Inter Sample Disc	0
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST
Start Pin Edge Typ	RISE
Stop Pin	TMU_SP
Stop Pin Edge Typ	FALL
RDI	
Mode	BURST
Pattern Label	tmu_rdi_test_patt
Port Label	tmu_port
Vector Offset1	4
Vector Offset2	4
Limits	...

RDI::
An example setup to do pin to pin delay measurement using datarate. Code is same as for Frequency measurement with the exception that the Application Type is set to P2P for the LogResult() calculation.
See Appendix Slide: c_tmu_base.cpp – LogResults()

The test method parameters are shown to correlate where the RDI inputs are coming from.
Test Method Parameters-
Application Type: P2P – This sets what calculation type will be performed on the measurements.
Start Pin: TMU_ST – set to pin where t1 edge will be
Start Pin Edge Type: RISE or FALL – Start pin reference edge.
Stop Pin: TMU_SP – set to pin where t2 edge will be
Stop Pin Edge Type: RISE or FALL – Stop pin edge.
Number of Shots: 1 – P2P only requires one TMU execution

TMU Programming – Pin to Pin Delay Measurement

- CalcPin2PinDelay() takes time stamps and performs calculations
- Requires a start pin and a stop pin to be specified in the parameters
- Calculates delta between Stop pin time stamps and the Start pin time stamps
- Number of shots should be 1 for Pin to pin Delay measurement
- Edge Mode expected is RISE or FALL and can be different between the start and stop pin

```
void tmu_base::CalcPin2PinDelay(string sname, int site)
{
        size_t pin_idx;
        int arm_idx, shot_idx, smp_idx;
        double result;
        ARRAY_D events1[3];
        ARRAY_D events2[3];
        map<string,ARRAY_D>::iterator it1;
        map<string,ARRAY_D>::iterator it2;
        cerr << endl << "Calculating \"PIN TO PIN DELAY\" Results for Site[" << site << "]" << endl << endl;

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it1 = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        it2 = m_mapStopEvents[shot_idx].find(m_vecStopPins[pin_idx]);
                        if(it1 != m_mapStartEvents[shot_idx].end() && it2 != m_mapStopEvents[shot_idx].end()) {
                                events1[shot_idx] = it1->second;
                                events2[shot_idx] = it2->second;
                                result = 0.0;
                                for(arm_idx=0; arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                        ARRAY_D arms1(m_iNumSamples);
                                        ARRAY_D arms2(m_iNumSamples);
                                        for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {
                                                arms1[smp_idx] = events1[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                                arms2[smp_idx] = events2[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                                result += (arms2[smp_idx]-arms1[smp_idx]);
                                        }
                                        result /= m_iNumSamples;
                                }
                                result /= (double)m_iNumMeasurementArms;

                                //cout << "Pin2PinDelay=" << result << endl;
                                JudgeAndLog(m_vecStopPins[pin_idx], "pin2pin", result, site);
                        } else {

                        }
                }
        }
}
```


This slide shows an excerpt of the result calculation code in the example program. The CalcPin2PinDelay function expects a number of shots parameter of 1 since only one TMU setup and execution is needed (Code does have a for loop for number of shots but should only go through this loop once). It delta between the stop pin and start pin time stamps. The delta is average across the number of samples. Finally if multiple ARMS had been placed in pattern then the result average across each ARM samples is calculated. The delta result between the 2 pins is returned as the result.

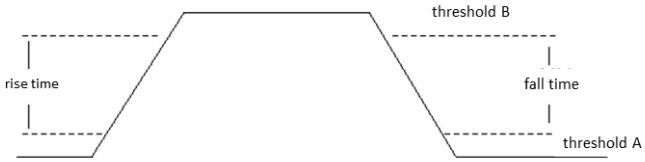
Code is similar to the CalcPulseWidth()

TMU Rise/Fall Time Measurement

ADVANTEST

Rise/Fall Time Measurement

- Requirement is to measure the rise time and the fall time between the two specified thresholds with TMU
- The measurement needs one arming
- Two level thresholds (Voltage threshold A and voltage threshold B) are required. The pattern is executed twice with the two different level thresholds.
- **NOTE: Cannot capture data for both thresholds in the same pattern execution**
- The rise time and the fall time are calculated from the subtraction of the two execution results. This is indicated in the following figure



References and Additional Information:
Reference 126960

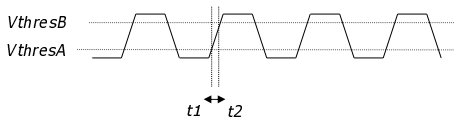
Rise/Fall Time Measurement

Measurement principle:

- Run pattern two times with different threshold voltage
- Take two timestamps t1, t2 of the same type of event

TMU setup specifics:

- Keep clock alive is necessary if it's PLL circuit



To measure the pin to pin skew or delay, place arming at exactly same time on both pins.
It is set by timing equations and arming cycle number in test pattern file.
After measured both events, calculate time difference on both pins.
one falling edges tf1.
When pre-scalar is used, the first edge is always discarded.
In case, the very first event after arming needs to be measures, set pre-scalar value to 1 which ignores pre-scalar function.

RDI: TMU Programming – Rise/Fall Time Measurement

```
RDI_INIT();
ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(m_sSuite);
        cerr << "Test Suite: " << m_sSuite << endl;
        CONNECT(); // For RISE/FALL measurements without DIB

             if(m_iNumShots == 2) {
                    RDI_BEGIN(m_rdiMode);
                          rdi.port(m_sPortLabel).tmu(2)
                                                .label(m_sPattLabel)
                                                .pin(m_sStartPins)
                                                .measNum(m_iNumMeasurementArms)
                                                .insertVec(m_iVecOffset[1])
                                                .insertVec(m_iVecOffset[2])
                                                .measMode(m_eStartEdgeTypeRDI)
                                                .dataRate(m_dDataRate)
                                                .vth(m_dThresholdB)
                                                .samples(m_iNumSamples)
                                                .initDiscard(m_iInitialDiscard)
                                                .cont();

                          rdi.port(m_sPortLabel).tmu(2).execute();
                          rdi.port(m_sPortLabel).wait(5e-3);
                    RDI_END();
               }

        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(curr_site);
if(tmu_ok) {
       LogResults(m_sSuite, curr_site);
} else {
       cerr << "TEST SUITE: \"" << m_sSuite << "\", SITE[" << curr_site << "] ***
}
return;
```

Second Shot requires voltage Threshold B to be set up

Test Method	tmu_ppt_tml.t_tmu_rdi_template.tmu_rdi_template
Parameters	...
Exec	
Setup	
Application Type	RISE
Number Of Measu	1
Samples Per Meas	4
DataRate	25[MHz]
Prescaler	1
Inter Sample Disc	0
Initial Discard	0
Number Of Shots	2
Threshold A	0.75[V]
Threshold B	2.25[V]
Pins	
Start Pin	TMU_ST
Start Pin Edge Typ	RISE
Stop Pin	
Stop Pin Edge Typ	NONE
RDI	
Mode	BURST
Pattern Label	tmu_rdi_test_patt
Port Label	tmu_port
Vector Offset1	4
Vector Offset2	4
Limits	...

See Appendix Slide: c_tmu_base.cpp – LogResults()

RDI::
An example setup to do Rise or Fall time measurement using datarate
The test method parameters are shown to correlate where the RDI inputs are coming from.
Test Method Parameters-
Application Type: RISE – This sets what calculation type will be performed on the measurements.
See Appendix Slide: c_tmu_base.cpp – LogResults()

Start Pin: TMU_ST – set to pin where t1 edge will be
Start Pin Edge Type: RISE or FALL – Start pin reference edge.
Number of Shots: 2 – RISE or FALL time measurement requires two TMU executions. One for each voltage threshold.
Threshold A: example shows setting to 25% point
Threshold B: example shows setting to 75% point

TMU Programming – Rise/Fall Time Measurement

- CalcRiseFall() takes time stamps and performs calculations
- Requires Voltage Threshold A and B to be specified in the parameters
- Calculates delta between Voltage Threshold A time stamps and the Voltage Threshold B time stamps
- Number of shots should be 2 for Rise/Fall time measurement for each voltage threshold
- Edge Mode expected is RISE or FALL

```
void tmu_base::CalcRiseFall(string sname, int site)
{
        size_t pin_idx;
        int shot_idx;
        double result;
        ARRAY_D events[3];

        map<string,ARRAY_D>::iterator it;
        cerr << endl << "Calculating \"RISE/FALL\" Results for Site[" << site << "]" << endl << endl;

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(it != m_mapStartEvents[shot_idx].end()) {
                                events[shot_idx] = it->second;
                        }
                }

                events[0].resize(m_iNumMeasurementArms*m_iNumSamples);

                for(int smp_idx=0; smp_idx<events[0].size(); smp_idx++) {
                        events[0][smp_idx] = events[2][smp_idx] - events[1][smp_idx];
                }
                Wave<double> wav(events[0]);
                result = wav.mean();
                //cout << "RISE/FALL=" << (result) << endl;
                if(m_eStartEdgeTypeAPI == TMU::RISE )
                        JudgeAndLog(m_vecStartPins[pin_idx], "rise", result, site);
                if(m_eStartEdgeTypeAPI == TMU::FALL )
                        JudgeAndLog(m_vecStartPins[pin_idx], "fall", result, site);

        }
}
```


This slide shows an excerpt of the result calculation code in the example program. The CalcRiseFall function expects a number of shots parameter of 2 since a TMU setup and execution is needed for both the voltage threshold A and the voltage threshold B. The delta between the voltage threshold time stamps is calculated. The delta is average across the number of samples.

SUMMARY

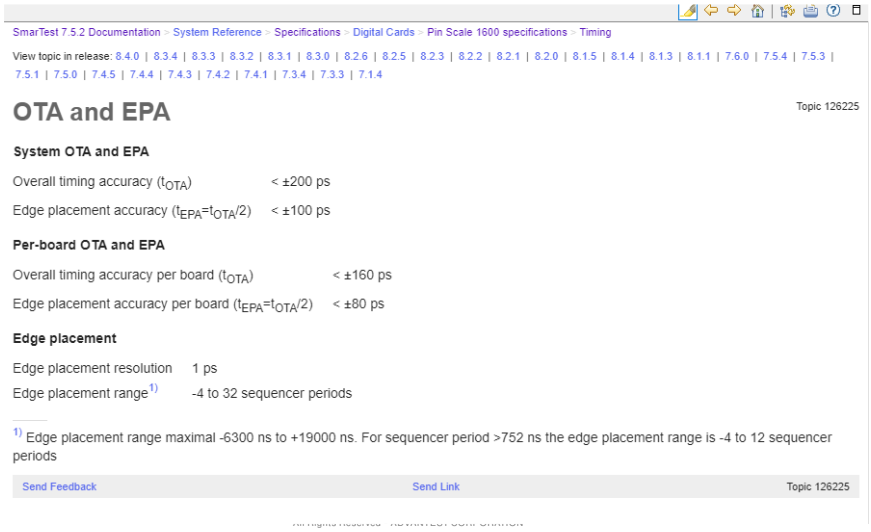
- The TMU (Time Measurement Unit) per pin is integrated into the test processor of the Smart Scale cards.
- Fast and easy way to accurately measure the period (frequency), pin to pin delay, pulse width, and rise/fall time of periodic and non-periodic output signals.
- The TMU measures the time from the ARM point to the event to be captured.
- Events are defined by a slope (rising, falling or both) and by a voltage threshold
- TMU measurements can be done on many pins in parallel since every channel has its own TMU
- Programming the TMU shown is SmartRDI
- Appendix contains details of MAPI programming

Frequently Asked Questions

ADVANTEST

FAQ

Question: Slide 4 – What does EPA in the TMU specifications refer to?



SmarTest 7.5.2 Documentation > System Reference > Specifications > Digital Cards > Pin Scale 1600 specifications > Timing

View topic in release: 8.4.0 | 8.3.4 | 8.3.3 | 8.3.2 | 8.3.1 | 8.3.0 | 8.2.6 | 8.2.5 | 8.2.3 | 8.2.2 | 8.2.1 | 8.2.0 | 8.1.5 | 8.1.4 | 8.1.3 | 8.1.1 | 7.6.0 | 7.5.4 | 7.5.3 | 7.5.1 | 7.5.0 | 7.4.5 | 7.4.4 | 7.4.3 | 7.4.2 | 7.4.1 | 7.3.4 | 7.3.3 | 7.1.4

OTA and EPA

Topic 126225

System OTA and EPA

Overall timing accuracy (t_{OTA})	< ±200 ps
Edge placement accuracy (t_{EPA}=t_{OTA}/2)	< ±100 ps

Per-board OTA and EPA

Overall timing accuracy per board (t_{OTA})	< ±160 ps
Edge placement accuracy per board (t_{EPA}=t_{OTA}/2)	< ±80 ps

Edge placement

Edge placement resolution	1 ps
Edge placement range[1)]	-4 to 32 sequencer periods

[1)] Edge placement range maximal -6300 ns to +19000 ns. For sequencer period >752 ns the edge placement range is -4 to 12 sequencer periods

Send Feedback Send Link Topic 126225

FAQ

Question: Slide 10 – What error code occurs if pulse width is less than the 5.5nS?

As the slide indicates the edge will not be captured. If there are enough events following the missed edge that are captured to satisfy the number of samples required, then no error message will occur. The result calculated from the captured events would show an invalid value that would require debug by the user.

If missing an expected edge causing the condition where not enough samples are captured, then an error will occur. In the example RDI code, if a TMU error occurs the array of time stamps returned by rdi.id().getStamp() will have a size of zero and the getResultCache() function checks the number of captured events against what is expected and will force an error message “!!!FAILED TO PROCESS TMU MEASURE RESULTS!!!” if they do not agree.

FAQ

Question: What error codes are there?

For MAPI you can interrogate for TMU errors as shown to right.

In RDI there is no method to interrogate for TMU codes. In the example RDI code, if a TMU error occurs the array of time stamps returned by rdi.id().getStamp() will have a size of zero and the getResultCache() function will check this against expected size.

View topic in release: 7.5.3 | 7.3.2 Reference 145798

TMU_RESULTS.getPinErrorState()

Syntax

```
UINT32 getPinErrorState(STRING& pin, INT site = 0);
void getPinErrorState(STRING_VECTOR& pins, std::vector<UINT32>& errors, INT site = 0);
```

This function returns the TMU error state of the specified pins. The state is returned as an integer that is to be decoded as the sum of one or more TMU::ERROR _TYPE enums in the following table:

Enum definition	Hexadecimal value	Decimal value	Error state meaning
TMU::NO_OVERRUN	0x00000000	0	No errors.
TMU::VDL_OVERRUN	0x00000001	1	The sample rate of the TMU is exceeded.
TMU::ARM_OVERRUN	0x00000002	2	The TMU was re-armed before all the requested samples had been taken in the preceding measurement run.
TMU::DEGLITCH_OVERRUN	0x00000004	4	The prescaler is programmed incorrectly.
TMU::OUT_OF_MEMORY	0x00000008	8	The embedded processor has run out of memory.
TMU::NOT_FINISHED	0x00000010	16	The measurement is still running.
TMU::LESS_SAMPLES_THAN_REQUESTED	0x00008000	32768	TMU sample count is smaller than total sample count in pattern.
TMU::SAMPLE_ACQUISITION_ERROR	0x00004000	16384	No sampling data available due to lost samples. This typically occurs in combination with VDL_OVERRUN or DEGLITCH_OVERRUN.
TMU::UNEXPECTED_TERMINATION	0x00002000	8000	Invalid pattern setup. EVENT_TMU_MEAS is not present.
TMU::ARMING_EVENT_MISSING	0x00001000	4000	Invalid pattern setup. EVENT_TMU_START is not present.
TMU::MEAS_ID_TABLE_INVALID	0x00000800	2048	Invalid pattern setup. No EVENT_TMU_MEAS object with mode END_MEAS is present.
TMU::LAST_EVENT_NOT_MEAS_END	0x00000400	1024	
TMU::RESULT_FIFO_OVERFLOW_STATUS	0x00000020	32	

Appendix

ADVANTEST

c_tmu_base.cpp – LogResults()

```
void tmu_base::LogResults(string sname, int site)
{
        int pin_idx, shot_idx, num_pins;
        map<string,ARRAY_D>::iterator st, sp;
        for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                num_pins = m_vecStartPins.size();
                for(pin_idx=0; pin_idx<num_pins; pin_idx++) {
                        st = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(st != m_mapStartEvents[shot_idx].end()) {
                                cerr << "\tShot Number[" << shot_idx << "]: " << "Start Pin <" << st->first << "> Data = " << st->second <<  endl;
                        }
                }

                num_pins = m_vecStopPins.size();
                for(pin_idx=0; pin_idx<num_pins; pin_idx++) {
                        sp = m_mapStopEvents[shot_idx].find(m_vecStopPins[pin_idx]);
                        if(sp != m_mapStopEvents[shot_idx].end()) {
                                cerr << "\tShot Number[" << shot_idx << "]: " << "Stop  Pin <" << sp->first << "> Data = " << sp->second <<  endl;
                        }
                }
        }
        if(m_sAppType == "PERIOD") {
                CalcPeriod(sname, site);
        } else if(m_sAppType == "FREQUENCY") {
                CalcFrequency(sname, site);
        } else if((m_sAppType == "DUTYCYCLE") || (m_sAppType == "PW")){
                CalcDutyCycle(sname, site);
        } else if(m_sAppType == "P2P") {
                CalcPin2PinDelay(sname, site);
        } else if((m_sAppType == "RISE") || (m_sAppType == "FALL")){
                CalcRiseFall(sname, site);
        } else if(m_sAppType == "JITTER") {
                CalcJitter(sname, site);
        } else if(m_sAppType == "EYE") {
                CalcEye(sname, site);
        } else if(m_sAppType == "DRIFT") {
                CalcDrift(sname, site);
        } else {
                CalcNone(sname, site);
        }
        ON_LAST_INVOCATION_BEGIN();
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        m_mapStartEvents[shot_idx].clear();
                        m_mapStopEvents[shot_idx].clear();
                }

                cerr << endl;
        ON_LAST_INVOCATION_END();
}
```


From c_tmu_base.cpp/.hpp

LogResults method:

```
void tmu_base::LogResults(string sname, int site)
{
        int pin_idx, shot_idx, num_pins;

        map<string,ARRAY_D>::iterator st, sp;

        for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                num_pins = m_vecStartPins.size();
                for(pin_idx=0; pin_idx<num_pins; pin_idx++) {
                        st =
m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(st !=
m_mapStartEvents[shot_idx].end()) {
                                cerr << "\tShot
Number[" << shot_idx << "]: " << "Start Pin <" << st->first << "> Data = " << st-
>second <<  endl;
                        }
```

```
                                }

                                num_pins = m_vecStopPins.size();
                                for(pin_idx=0; pin_idx<num_pins; pin_idx++) {
                                        sp =
m_mapStopEvents[shot_idx].find(m_vecStopPins[pin_idx]);
                                        if(sp !=
m_mapStopEvents[shot_idx].end()) {
                                                cerr << "\tShot
Number[" << shot_idx << "]: " << "Stop  Pin <" << sp->first << "> Data = " << sp-
>second <<  endl;
                                        }
                                }
                }

                if(m_sAppType == "PERIOD") {
                        CalcPeriod(sname, site);
                } else if(m_sAppType == "FREQUENCY") {
                        CalcFrequency(sname, site);
                } else if((m_sAppType == "DUTYCYCLE") || (m_sAppType == "PW")){
                        CalcDutyCycle(sname, site);
                } else if(m_sAppType == "P2P") {
                        CalcPin2PinDelay(sname, site);
                } else if((m_sAppType == "RISE") || (m_sAppType == "FALL")){
                        CalcRiseFall(sname, site);
                } else if(m_sAppType == "JITTER") {
                        CalcJitter(sname, site);
                } else if(m_sAppType == "EYE") {
                        CalcEye(sname, site);
                } else if(m_sAppType == "DRIFT") {
                        CalcDrift(sname, site);
                } else {
                        CalcNone(sname, site);
                }

                ON_LAST_INVOCATION_BEGIN();
                        for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++)
{
                                m_mapStartEvents[shot_idx].clear();
                                m_mapStopEvents[shot_idx].clear();
                        }

                        cerr << endl;
                ON_LAST_INVOCATION_END();
}
```

C_tmu_base.hpp

```
#ifndef _C_TMU_BASE_HPP_
#define _C_TMU_BASE_HPP_

#include <sstream>
#include <string>
#include <iostream>
#include <map>

#include "testmethod.hpp"
#include "mapi.hpp"
#include "rdi.hpp"

using namespace std;

class tmu_base: public testmethod::TestMethod
{
private:
                //      APP_NONE          = 0x00000000,
                //      APP_PERIOD        = 0x00000001,
                //      APP_FREQUENCY     = 0x00000002,
                //      APP_PW            = 0x00000010,
                //      APP_RISE          = 0x00000100,
                //      APP_FALL          = 0x00000200,
                //      APP_JITTER        = 0x00001000,
                //      APP_EYE           = 0x00010000,
                //      APP_DRIFT         = 0x00100000,

                void CalcPeriod(string sname, int site);
                void CalcFrequency(string sname, int site);
                void CalcDutyCycle(string sname, int site);
                void CalcPulseWidth(string sname, int site);
                void CalcPin2PinDelay(string sname, int site);
                void CalcRiseFall(string sname, int site);
                void CalcJitter(string sname, int site);
                void CalcEye(string sname, int site);
                void CalcDrift(string sname, int site);
                void CalcNone(string sname, int site);

                bool Log(string sPin, string sname, double dValue, int site, string unit);
                bool JudgeAndLog(string sPin, string sname, double dValue, int site);
                double PostScaleResult(double val);
                double ScaleResult(double val, LIMIT &lim);

protected:

                STRING_VECTOR m_vecStartPins;
                STRING_VECTOR m_vecStopPins;

                map<string,ARRAY_D> m_mapStartEvents[3];
                map<string,ARRAY_D> m_mapStopEvents[3];

                testmethod::SpecValue specDataRate;
                testmethod::SpecValue specThresholdA;
                testmethod::SpecValue specThresholdB;
```


c_tmu_base.hpp

```
#ifndef _C_TMU_BASE_HPP_
#define _C_TMU_BASE_HPP_

#include <sstream>
#include <string>
#include <iostream>
#include <map>

#include "testmethod.hpp"
#include "mapi.hpp"
#include "rdi.hpp"

using namespace std;

class tmu_base: public testmethod::TestMethod
{
private:
                //      APP_NONE          = 0x00000000,
```

```
        //      APP_PERIOD        = 0x00000001,
        //      APP_FREQUENCY     = 0x00000002,
        //      APP_PW            = 0x00000010,
        //      APP_RISE          = 0x00000100,
        //      APP_FALL          = 0x00000200,
        //      APP_JITTER        = 0x00001000,
        //      APP_EYE           = 0x00010000,
        //      APP_DRIFT         = 0x00100000,

        void CalcPeriod(string sname, int site);
        void CalcFrequency(string sname, int site);
        void CalcDutyCycle(string sname, int site);
        void CalcPulseWidth(string sname, int site);
        void CalcPin2PinDelay(string sname, int site);
        void CalcRiseFall(string sname, int site);
        void CalcJitter(string sname, int site);
        void CalcEye(string sname, int site);
        void CalcDrift(string sname, int site);
        void CalcNone(string sname, int site);

        bool Log(string sPin, string sname, double dValue, int site, string
unit);
        bool JudgeAndLog(string sPin, string sname, double dValue, int site);
        double PostScaleResult(double val);
        double ScaleResult(double val, LIMIT &lim);

protected:

        STRING_VECTOR m_vecStartPins;
        STRING_VECTOR m_vecStopPins;

        map<string,ARRAY_D> m_mapStartEvents[3];
        map<string,ARRAY_D> m_mapStopEvents[3];

        testmethod::SpecValue specDataRate;
        testmethod::SpecValue specThresholdA;
        testmethod::SpecValue specThresholdB;

        string m_sStartPins, m_sStartEdge;
        string m_sStopPins, m_sStopEdge;
        string m_sAppType;
        string m_sUnit;

        TMU::ApplicationType m_eAppType;
```

```
        TMU::EdgeType m_eStartEdgeTypeAPI;
        TMU::EdgeType m_eStopEdgeTypeAPI;

        TA::TMU_EDGE_TYPE m_eStartEdgeTypeRDI;
        TA::TMU_EDGE_TYPE m_eStopEdgeTypeRDI;

        TMU::DATARATE_DEPENDENT_VALUES_TYPE m_drSettings;

        bool m_bUseStopPin;
        bool m_bUseDataRate;

        int m_iTestNumber;
        int m_iNumTmuTasks;
        int m_iNumSamples;
        int m_iNumShots;
        int m_iNumMeasurementArms;
        int m_iPrescaler;
        int m_iInterDiscard;
        int m_iInitialDiscard;

        UINT64 m_iDataRate;

        double m_dThresholdA, m_dThresholdB;
        double m_dDataRate;

        double m_dLoLimit;
        double m_dHiLimit;

        bool useTesterDataFile;

        virtual void initialize();
        virtual void run();
        virtual void postParameterChange(const string&
parameterIdentifier);
        virtual const string getComment() const;

        void LogResults(string sname, int site);

};

#endif /* _C_TMU_BASE_HPP_ */
```

C_tmu_base.cpp

```
#include "c_tmu_base.hpp"

void tmu_base::initialize()
{
                m_iNumTmuTasks = 1;

//      APP_NONE          = 0x00000000,
//      APP_PERIOD        = 0x00000001,
//      APP_FREQUENCY     = 0x00000002,
//      APP_PW            = 0x00000010,
//      APP_RISE          = 0x00000100,
//      APP_FALL          = 0x00000200,
//      APP_JITTER        = 0x00001000,
//      APP_EYE           = 0x00010000,
//      APP_DRIFT         = 0x00100000,

                addParameter("Exec.Test Number",
                                                                "int",
                                                                &m_iTestNumber)
                                                                .setDefault("0")
                                                                .setComment("TMU Test Number");

                addParameter("Exec.Low Limit",
                                                                "double",
                                                                &m_dLoLimit)
                                                                .setDefault("0")
                                                                .setComment("TMU Test Low Limit");

                addParameter("Exec.High Limit",
                                                                "double",
                                                                &m_dHiLimit)
                                                                .setDefault("0")
                                                                .setComment("TMU Test High Limit");

                addParameter("Exec.Unit",
                                                                "string",
                                                                &m_sUnit)
                                                                .setComment("TMU Test Unit");

                addParameter("Setup.Application Type",
                                                                "string",
                                                                &m_sAppType)
                                                                .setDefault("NONE")
                                                                .setOptions("NONE:PERIOD:FREQUENCY:DUTYCYCLE:PW:P2P:RISE:FALL:JITTER:EYE:DRIFT")
                                                                .setComment("TMU Measurement Type");

                addParameter("Setup.Number Of Measurement ARMS",
                                                                "int",
                                                                &m_iNumMeasurementArms)
                                                                .setDefault("1")
                                                                .setOptions("1:2")
                                                                .setComment("Number of \"ARMS\" in pattern");
                addParameter("Setup.Samples Per Measurement",
                                                                "int",
```


c_tmu_base.cpp

```
#include "c_tmu_base.hpp"

void tmu_base::initialize()
{
                m_iNumTmuTasks = 1;

//      APP_NONE          = 0x00000000,
//      APP_PERIOD        = 0x00000001,
//      APP_FREQUENCY     = 0x00000002,
//      APP_PW            = 0x00000010,
//      APP_RISE          = 0x00000100,
//      APP_FALL          = 0x00000200,
//      APP_JITTER        = 0x00001000,
//      APP_EYE           = 0x00010000,
//      APP_DRIFT         = 0x00100000,

                addParameter("Exec.Test Number",
                                                                "int",
```

```
                                                    &m_iTestNumber)
                                                    .setDefault("0")
                                                    .setComment("TMU
Test Number");

                addParameter("Exec.Low Limit",
                                                    "double",
                                                    &m_dLoLimit)
                                                    .setDefault("0")
                                                    .setComment("TMU
Test Low Limit");

                addParameter("Exec.High Limit",
                                                    "double",
                                                    &m_dHiLimit)
                                                    .setDefault("0")
                                                    .setComment("TMU
Test High Limit");

                addParameter("Exec.Unit",
                                                    "string",
                                                    &m_sUnit)
                                                    .setComment("TMU
Test Unit");

                addParameter("Setup.Application Type",
                                                    "string",
                                                    &m_sAppType)
                                                    .setDefault("NONE")

                 .setOptions("NONE:PERIOD:FREQUENCY:DUTYCYCLE:PW:P2P:RISE:F
ALL:JITTER:EYE:DRIFT")
                                                    .setComment("TMU
Measurement Type");

                addParameter("Setup.Number Of Measurement ARMS",
                                                    "int",

&m_iNumMeasurementArms)
                                                    .setDefault("1")
                                                    .setOptions("1:2")
                                                    .setComment("Number
of \"ARMS\" in pattern");

                addParameter("Setup.Samples Per Measurement",
```

```
                                                    "int",
                                                    &m_iNumSamples)
                                                    .setDefault("1")
                                                    .setComment("Number
of samples for each measurement");

                addParameter("Setup.DataRate",
                                                    "SpecValue",
                                                    &specDataRate)
                                                    .setDefault("25[MHz]")

                 .setComment("Maximum expected datarate");

                m_bUseDataRate = false;

                addParameter("Setup.Prescaler",
                                                    "int",
                                                    &m_iPrescaler)
                                                    .setDefault("1")

                 .setOptions("1:2:3:4:5:6:7:8:9:10:11:12:13:14:15:16")
                                                    .setEnabled(true)

                 .setComment("Prescaler, integer from 2 to 16.");

                addParameter("Setup.Inter Sample Discard",
                                                    "int",
                                                    &m_iInterDiscard)
                                                    .setDefault("0")
                                                    .setEnabled(true)
                                                    .setComment("Events
to be discarded between samples");

                addParameter("Setup.Initial Discard",
                                                    "int",
                                                    &m_iInitialDiscard)
                                                    .setDefault("0")
                                                    .setComment("Events
to be discarded after arming");

                addParameter("Setup.Number Of Shots",
                                                    "int",
                                                    &m_iNumShots)
                                                    .setDefault("1")
                                                    .setOptions("1:2")
```

```
                                                    .setComment("Number
of TMU Shots (Measurements)");

                addParameter("Setup.Threshold A",
                                                    "SpecValue",
                                                    &specThresholdA)
                                                    .setDefault("0.75[V]")
                                                    .setComment("Voltage
threshold for first shot");

                addParameter("Setup.Threshold B",
                                                    "SpecValue",
                                                    &specThresholdB)
                                                    .setDefault("2.25[V]")
                                                    .setEnabled(false)
                                                    .setComment("Voltage
threshold for second shot");

                addParameter("Pins.Start Pin",
                                                    "PinString",
                                                    &m_sStartPins)
                                                    .setDefault("")
                                                    .setComment("Start Pin
for TMU Measurement");

                addParameter("Pins.Start Pin Edge Type",
                                                    "string",
                                                    &m_sStartEdge)
                                                    .setDefault("NONE")

                 .setOptions("NONE:RISE:FALL:RISE_FALL:FALL_RISE")
                                                    .setComment("Start Pin
Edge Type for TMU Measurement");

                addParameter("Pins.Stop Pin",
                                                    "PinString",
                                                    &m_sStopPins)
                                                    .setDefault("")
                                                    .setEnabled(false)
                                                    .setComment("Stop Pin
for TMU Measurement");

                addParameter("Pins.Stop Pin Edge Type",
                                                    "string",
                                                    &m_sStopEdge)
```

```
                                                        .setDefault("NONE")

                .setOptions("NONE:RISE:FALL")
                                                        .setEnabled(false)
                                                        .setComment("Stop Pin
Edge Type for TMU Measurement");

                m_bUseStopPin = false;
}

void tmu_base::run()
{

}

void tmu_base::postParameterChange(const string& parameterIdentifier)
{
                if( parameterIdentifier == "Setup.Number Of Shots") {
                                string val;
                                val = getParameter("Setup.Number Of
Shots").getValueAsString();
                                if(val == "1")
                                                getParameter("Setup.Threshold
B").setEnabled(false);
                                else
                                                getParameter("Setup.Threshold
B").setEnabled(true);
                }

                if( parameterIdentifier == "Setup.DataRate") {
                                m_dDataRate =
specDataRate.getValueAsTargetUnit("Hz");
                                if(m_dDataRate > 0) {

                getParameter("Setup.Prescaler").setEnabled(false);
                                                getParameter("Setup.Inter Sample
Discard").setEnabled(false);

                                                m_iDataRate = (int)m_dDataRate;
                                                m_bUseDataRate = true;
                                }
                                else {

                getParameter("Setup.Prescaler").setEnabled(true);
                                                getParameter("Setup.Inter Sample
```

```
Discard").setEnabled(true);

                                                m_iDataRate = 0;
                                                m_bUseDataRate = false;
                                }
                }

                if( parameterIdentifier == "Setup.Application Type") {
                                string val;
                                val = getParameter("Setup.Application
Type").getValueAsString();
                                if(val != "P2P") {
                                                m_bUseStopPin = false;
                                                m_sStopPins = "";
                                                m_vecStopPins.clear();

                                                getParameter("Pins.Stop
Pin").setValueAsString(m_sStopPins);
                                                getParameter("Pins.Stop
Pin").setEnabled(false);
                                                getParameter("Pins.Stop Pin Edge
Type").setEnabled(false);
                                                getParameter("Pins.Stop Pin Number
Of Measurements").setEnabled(false);
                                }
                                else {
                                                m_bUseStopPin = true;

                                                getParameter("Pins.Stop
Pin").setEnabled(true);
                                                getParameter("Pins.Stop Pin Edge
Type").setEnabled(true);
                                                getParameter("Pins.Stop Pin Number
Of Measurements").setEnabled(true);
                                }
                }

                if( parameterIdentifier == "Pins.Start Pin") {
                                string val;
                                val = getParameter("Pins.Start
Pin").getValueAsString();
                                m_vecStartPins =
PinUtility.getDigitalPinNamesFromPinList(val,
```

```
                TM::I_PIN|TM::O_PIN|TM::IO_PIN,


                false,false,PIN_UTILITY::DEFINITION_ORDER);
                }

                if( parameterIdentifier == "Pins.Stop Pin") {
                                string val;
                                val = getParameter("Pins.Stop
Pin").getValueAsString();
                                m_vecStopPins =
PinUtility.getDigitalPinNamesFromPinList(val,


                TM::I_PIN|TM::O_PIN|TM::IO_PIN,


                false,false,PIN_UTILITY::DEFINITION_ORDER);
                }

                if( parameterIdentifier == "Pins.Start Pin Edge Type") {
                                string val;
                                val = getParameter("Pins.Start Pin Edge
Type").getValueAsString();
                                if(val == "NONE") {
                                                m_eStartEdgeTypeAPI = TMU::NONE;
                                                m_eStartEdgeTypeRDI = TA::NONE;
                                } else if(val == "RISE") {
                                                m_eStartEdgeTypeAPI = TMU::RISE;
                                                m_eStartEdgeTypeRDI = TA::RISE;
                                } else if(val == "FALL") {
                                                m_eStartEdgeTypeAPI = TMU::FALL;
                                                m_eStartEdgeTypeRDI = TA::FALL;
                                } else if(val == "RISE_FALL") {
                                                m_eStartEdgeTypeAPI =
TMU::RISE_FALL;
                                                m_eStartEdgeTypeRDI = TA::RISE_FALL;
                                } else if(val == "FALL_RISE") {
                                                m_eStartEdgeTypeAPI =
TMU::FALL_RISE;
                                                m_eStartEdgeTypeRDI = TA::RISE_FALL;
                                }
                }

                if( parameterIdentifier == "Pins.Stop Pin Edge Type") {
```

```
                                string val;
                                val = getParameter("Pins.Stop Pin Edge
Type").getValueAsString();
                                if(val == "NONE") {
                                                m_eStopEdgeTypeAPI = TMU::NONE;
                                                m_eStopEdgeTypeRDI = TA::NONE;
                                } else if(val == "RISE") {
                                                m_eStopEdgeTypeAPI = TMU::RISE;
                                                m_eStopEdgeTypeRDI = TA::RISE;
                                } else if(val == "FALL") {
                                                m_eStopEdgeTypeAPI = TMU::FALL;
                                                m_eStopEdgeTypeRDI = TA::FALL;
                                } else if(val == "RISE_FALL") {
                                                m_eStopEdgeTypeAPI =
TMU::RISE_FALL;
                                                m_eStopEdgeTypeRDI = TA::RISE_FALL;
                                }  else if(val == "FALL_RISE") {
                                                m_eStopEdgeTypeAPI =
TMU::FALL_RISE;
                                                m_eStopEdgeTypeRDI = TA::RISE_FALL;
                                }
                }

                m_dThresholdA=specThresholdA.getValueAsTargetUnit("V");

                if(getParameter("Setup.Threshold B").isEnabled()) {

                m_dThresholdB=specThresholdB.getValueAsTargetUnit("V");
                } else {
                                m_dThresholdB=0.0;
                }

}

const string tmu_base::getComment() const
{
                string comment = " please add your comment for this method.";
                return comment;
}

void tmu_base::LogResults(string sname, int site)
{
                int pin_idx, shot_idx, num_pins;

                map<string,ARRAY_D>::iterator st, sp;
```

```
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                                num_pins = m_vecStartPins.size();
                                for(pin_idx=0; pin_idx<num_pins; pin_idx++) {
                                                st =
m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                                                if(st !=
m_mapStartEvents[shot_idx].end()) {
                                                                cerr << "\tShot
Number[" << shot_idx << "]: " << "Start Pin <" << st->first << "> Data = " << st-
>second <<  endl;
                                                }
                                }

                                num_pins = m_vecStopPins.size();
                                for(pin_idx=0; pin_idx<num_pins; pin_idx++) {
                                                sp =
m_mapStopEvents[shot_idx].find(m_vecStopPins[pin_idx]);
                                                if(sp !=
m_mapStopEvents[shot_idx].end()) {
                                                                cerr << "\tShot
Number[" << shot_idx << "]: " << "Stop  Pin <" << sp->first << "> Data = " << sp-
>second <<  endl;
                                                }
                                }
                }

                if(m_sAppType == "PERIOD") {
                                CalcPeriod(sname, site);
                } else if(m_sAppType == "FREQUENCY") {
                                CalcFrequency(sname, site);
                } else if((m_sAppType == "DUTYCYCLE") || (m_sAppType == "PW")){
                                CalcDutyCycle(sname, site);
                } else if(m_sAppType == "P2P") {
                                CalcPin2PinDelay(sname, site);
                } else if((m_sAppType == "RISE") || (m_sAppType == "FALL")){
                                CalcRiseFall(sname, site);
                } else if(m_sAppType == "JITTER") {
                                CalcJitter(sname, site);
                } else if(m_sAppType == "EYE") {
                                CalcEye(sname, site);
                } else if(m_sAppType == "DRIFT") {
                                CalcDrift(sname, site);
                } else {
                                CalcNone(sname, site);
```

```
		}

		ON_LAST_INVOCATION_BEGIN();
			for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++)
{
				m_mapStartEvents[shot_idx].clear();
				m_mapStopEvents[shot_idx].clear();
			}

			cerr << endl;
		ON_LAST_INVOCATION_END();
}

// private methods
void tmu_base::CalcNone(string sname, int site)
{
		cerr << endl << "*** Setup.Application Type = \"NONE\" ***" <<
endl << endl;
}

void tmu_base::CalcPeriod(string sname, int site)
{
		size_t pin_idx;
		int arm_idx, shot_idx, smp_idx;
		double result;
		ARRAY_D events[3];

		map<string,ARRAY_D>::iterator it;
		cerr << endl << "Calculating \"PERIOD\" Results for Site[" << site <<
"]" << endl << endl;

		for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
			for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++)
{
				it =
m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
				if(it !=
m_mapStartEvents[shot_idx].end()) {
					events[shot_idx] = it-
>second;
					result = 0.0;
					for(arm_idx=0;
arm_idx<m_iNumMeasurementArms; arm_idx++) {
						ARRAY_D
arms(m_iNumSamples);
```

```
                for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {

                arms[smp_idx] =
events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                                                        }

                Wave<double> wav(arms);
                                                                        int len =
wav.getSize();
                                                                        double
val;
                                                                        val =
wav.differentiate().select(1,1, len-1).mean();
                                                                        val =
PostScaleResult(val);
                                                                        result +=
val;
                                                                }
                                                                result /=
(double)m_iNumMeasurementArms;
                                                                //cout << "Period=" <<
result << endl;

                JudgeAndLog(m_vecStartPins[pin_idx], "period", result, site);
                                                } else {

                                                }
                                        }
                }
}

void tmu_base::CalcFrequency(string sname, int site)
{
                size_t pin_idx;
                int arm_idx, shot_idx, smp_idx;
                double result;
                ARRAY_D events[3];

                map<string,ARRAY_D>::iterator it;
                cerr << endl << "Calculating \"FREQUENCY\" Results for Site[" << site
<< "]" << endl << endl;

                for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++)
```

```
{
                                                it =
m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                                                if(it !=
m_mapStartEvents[shot_idx].end()) {
                                                                events[shot_idx] = it-
>second;
                                                                result = 0.0;
                                                                for(arm_idx=0;
arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                                                                ARRAY_D
arms(m_iNumSamples);

                for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {

                arms[smp_idx] =
events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                                                                }

                Wave<double> wav(arms);
                                                                                int len =
wav.getSize();
                                                                                double
val;
                                                                                val =
wav.differentiate().select(1,1, len-1).mean();
                                                                                val =
PostScaleResult(val);
                                                                                result +=
val;
                                                                }
                                                                result /=
(double)m_iNumMeasurementArms;
                                                                //cout << "Frequency="
<< (1.0/result) << endl;

                JudgeAndLog(m_vecStartPins[pin_idx], "freq", 1.0/result, site);
                                                } else {

                                                }
                                }
                }
}

void tmu_base::CalcDutyCycle(string sname, int site)
```

```
{
		size_t pin_idx;
		int arm_idx, shot_idx, smp_idx;
		double result;
		double resPeriod;
		double resThi;
		double resTlo;
		ARRAY_D events[3];
		double prescale = 0.0;
		double interskip = 0.0;

		map<string,ARRAY_D>::iterator it;
		if(m_sAppType == "DUTYCYCLE") {
			cerr << endl << "Calculating \"DUTY CYCLE\" Results
for Site[" << site << "]" << endl << endl;
		}
		else {
			cerr << endl << "Calculating \"PulseWidth\" Results
for Site[" << site << "]" << endl << endl;
		}

		for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
			for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++)
{
				it =
m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
				if(it !=
m_mapStartEvents[shot_idx].end()) {
					events[shot_idx] = it-
>second;

					result = 0.0;
					resPeriod = 0.0;
					resThi = 0.0;
					resTlo = 0.0;
					for(arm_idx=0;
arm_idx<m_iNumMeasurementArms; arm_idx++) {
						ARRAY_D
arms(m_iNumSamples);
						double
period=0;
						double
thi=0;
						double
tlo=0;
```

```
                for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {

                arms[smp_idx] =
events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                                                        }

                                                                        // array is
setup !!!

                if(m_bUseDataRate) {

                prescale = (double)m_drSettings.prescaler;

                interskip = (double)m_drSettings.interdiscard;
                                                                        } else {

                prescale = (double)m_iPrescaler;

                interskip = (double)m_iInterDiscard;
                                                                        }

                                                                        for(int
i=0; i<(m_iNumSamples/3)*3; i+=3) {

                period = (arms[i+2]-arms[i]) / ((prescale+0.5)*(interskip+1)*2);

                thi = fmod(arms[i+1]-arms[i],period);

                tlo = fmod(arms[i+2]-arms[i+1],period);
                                                                        }

                                                                        resPeriod
+= period;
                                                                        resThi +=
thi;
                                                                        resTlo +=
tlo;
                                                                }
                                                                resPeriod /=
(double)m_iNumMeasurementArms;
                                                                resThi /=
(double)m_iNumMeasurementArms;
                                                                resTlo /=
(double)m_iNumMeasurementArms;
```

```
                                                result =  resThi /
resPeriod * 100;
                                                if(m_sAppType ==
"DUTYCYCLE") {

                JudgeAndLog(m_vecStartPins[pin_idx], "duty", result, site);

                Log(m_vecStartPins[pin_idx], "period", resPeriod, site, "ns");

                Log(m_vecStartPins[pin_idx], "thi", resThi, site, "ns");

                Log(m_vecStartPins[pin_idx], "tlo", resTlo, site, "ns");

                                                } else {        // must
be a PW measurement
                                                //
Determine if we are doing a "negative" pw measurement, or a positive pw
measurement

                if(m_eStartEdgeTypeAPI == TMU::RISE_FALL )

                JudgeAndLog(m_vecStartPins[pin_idx], "thi", resThi, site);

                if(m_eStartEdgeTypeAPI == TMU::FALL_RISE )

                JudgeAndLog(m_vecStartPins[pin_idx], "tlo", resTlo, site);
                                                }
                                        } else {
                                                cerr << "Error!" << endl;
                                        }
                        }
                }
}

void tmu_base::CalcPin2PinDelay(string sname, int site)
{

                size_t pin_idx;
                int arm_idx, shot_idx, smp_idx;
                double result;
                ARRAY_D events1[3];
                ARRAY_D events2[3];

                map<string,ARRAY_D>::iterator it1;
                map<string,ARRAY_D>::iterator it2;
```

```
		cerr << endl << "Calculating \"PIN TO PIN DELAY\" Results for Site["
<< site << "]" << endl << endl;

		for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
			for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++)
{
				it1 =
m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
				it2 =
m_mapStopEvents[shot_idx].find(m_vecStopPins[pin_idx]);
				if(it1 !=
m_mapStartEvents[shot_idx].end() && it2 != m_mapStopEvents[shot_idx].end()) {
					events1[shot_idx] = it1-
>second;
					events2[shot_idx] = it2-
>second;
					result = 0.0;
					for(arm_idx=0;
arm_idx<m_iNumMeasurementArms; arm_idx++) {
						ARRAY_D
arms1(m_iNumSamples);
						ARRAY_D
arms2(m_iNumSamples);

		for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {

		arms1[smp_idx] =
events1[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];

		arms2[smp_idx] =
events2[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];

		result += (arms2[smp_idx]-arms1[smp_idx]);
						}
						result /=
m_iNumSamples;
					}
					result /=
(double)m_iNumMeasurementArms;

					//cout <<
"Pin2PinDelay=" << result << endl;

		JudgeAndLog(m_vecStopPins[pin_idx], "pin2pin", result, site);
				} else {
```

```
                                        }
                                }
                        }
}

void tmu_base::CalcRiseFall(string sname, int site)
{
                        size_t pin_idx;
                        int shot_idx;
                        double result;
                        ARRAY_D events[3];

                        map<string,ARRAY_D>::iterator it;
                        cerr << endl << "Calculating \"RISE/FALL\" Results for Site[" << site <<
"]" << endl << endl;

                        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++)
{
                                        it =
m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                                        if(it !=
m_mapStartEvents[shot_idx].end()) {
                                                events[shot_idx] = it-
>second;
                                        }
                                }


                        events[0].resize(m_iNumMeasurementArms*m_iNumSamples);

                                for(int smp_idx=0; smp_idx<events[0].size();
smp_idx++) {
                                        events[0][smp_idx] =
events[2][smp_idx] - events[1][smp_idx];
                                }
                                Wave<double> wav(events[0]);
                                result = wav.mean();
                                //cout << "RISE/FALL=" << (result) << endl;
                                if(m_eStartEdgeTypeAPI == TMU::RISE )
                                        JudgeAndLog(m_vecStartPins[pin_idx],
"rise", result, site);
                                if(m_eStartEdgeTypeAPI == TMU::FALL )
                                        JudgeAndLog(m_vecStartPins[pin_idx],
```

```
"fall", result, site);

		}
}

void tmu_base::CalcJitter(string sname, int site)
{
		double result;

		cerr << endl << "Calculating \"JITTER\" Results for Site[" << site << "]"
<< endl << endl;

		// TODO: Calculate Result!!!
		for(size_t pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
			JudgeAndLog(m_vecStartPins[pin_idx], sname, result,
site);
		}
}

void tmu_base::CalcEye(string sname, int site)
{
		double result;

		cerr << endl << "Calculating \"EYE\" Results for Site[" << site << "]"
<< endl << endl;

		// TODO: Calculate Result!!!
		for(size_t pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
			JudgeAndLog(m_vecStartPins[pin_idx], sname, result,
site);
		}
}

void tmu_base::CalcDrift(string sname, int site)
{
		double result;

		cerr << endl << "Calculating \"DRIFT\" Results for Site[" << site << "]"
<< endl << endl;

		// TODO: Calculate Result!!!
		for(size_t pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
			JudgeAndLog(m_vecStartPins[pin_idx], sname, result,
site);
		}
```

```
}

bool tmu_base::Log(string sPin, string sname, double dValue, int site, string unit)
{
		bool result;

		LIMIT limit;
		TESTSET test;

   if((unit == "pS")  || (unit == "ps"))  dValue *= 1e12;
		else if((unit == "nS")  || (unit == "ns"))  dValue *= 1e9;
		else if((unit == "uS")  || (unit == "us"))  dValue *= 1e6;
		else if((unit == "mS")  || (unit == "ms"))  dValue *= 1e3;
		else if((unit == "KHz") || (unit == "KHz")) dValue /= 1e3;
		else if((unit == "MHz") || (unit == "MHz")) dValue /= 1e6;

		limit.low(TM::GE, dValue);
		limit.high(TM::LE, dValue);
		limit.unit(unit);

		test.testnumber(m_iTestNumber++);
		test.cont(true);
		test.resFmt("%10.3f").llmFmt("%10.3f").hlmFmt("%10.3f");

		result = test.judgeAndLog_ParametricTest(sPin, sname, limit,
dValue);

		return result;
}

bool tmu_base::JudgeAndLog(string sPin, string sname, double dValue, int site)
{
		bool result;

		LIMIT limit;
		TESTSET test;

		limit.low(TM::GE, m_dLoLimit);
		limit.high(TM::LE, m_dHiLimit);
		limit.unit(m_sUnit);

		test.testnumber(m_iTestNumber++);
		test.cont(true);
		test.resFmt("%10.3f").llmFmt("%10.3f").hlmFmt("%10.3f");
```

```
		result = test.judgeAndLog_ParametricTest(sPin, sname, limit,
ScaleResult(dValue, limit));

		return result;
}



double tmu_base::ScaleResult(double val, LIMIT &lim)
{
		int offline;

		GET_SYSTEM_FLAG("offline", &offline);

		   if((m_sUnit == "pS")  || (m_sUnit == "ps"))  val *= 1e12;
		else if((m_sUnit == "nS")  || (m_sUnit == "ns"))  val *= 1e9;
		else if((m_sUnit == "uS")  || (m_sUnit == "us"))  val *= 1e6;
		else if((m_sUnit == "mS")  || (m_sUnit == "ms"))  val *= 1e3;
		else if((m_sUnit == "KHz") || (m_sUnit == "KHz")) val /= 1e3;
		else if((m_sUnit == "MHz") || (m_sUnit == "MHz")) val /= 1e6;

if(!useTesterDataFile) {
		if(offline)
		{
				TM::COMPARE opl, oph;
				double  lo, hi;

				lim.get( opl, lo, oph, hi);

				if((opl != TM::NA) && (oph != TM::NA))
						val = (double)((lo+hi)/2.0);
				else if((opl == TM::NA) && (oph != TM::NA))
						val = (double)(hi-fabs(hi/2.0));
				else if((opl != TM::NA) && (oph == TM::NA))
						val = (double)(lo+ fabs(lo/2.0));
				else
						val = 0.0;
		}
}
		return val;
}

double tmu_base::PostScaleResult(double val)
{
		double result;
```

```
		double prescale;
		double interskip;

		if(m_bUseDataRate) {
				prescale = (double)m_drSettings.prescaler;
				interskip = (double)m_drSettings.interdiscard;
		} else {
				prescale = (double)m_iPrescaler;
				interskip = (double)m_iInterDiscard;
		}
		result = val/(prescale*(interskip+1.0));

		return result;
}
```

TMU Test Type Coding Examples

- Previously described using SmartRDI
- Following shows MAPI examples

MAPI TMU Frequency Measurement

ADVANTEST

setDataRate - MAPI

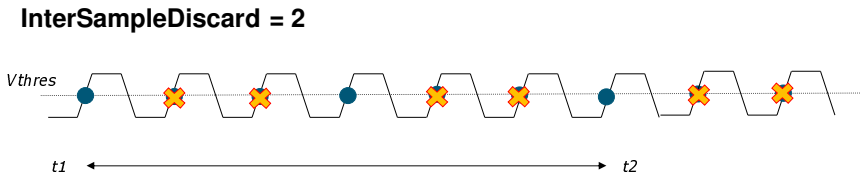
- Sets the maximum expected data rate for the TMU pin(s)
- TMU_PINS.setDatarate()
- SmarTest will calculate the proper values for prescaler and interdiscard.
- Interdiscard can be changed to a higher value. If a higher interdiscard is already set, it will not be changed to the calculated value. Interdiscard needs to be changed to a higher value for period and frequency measurement to get accurate results.

MAPI::
TMU_PINS.getDatarateDependendSettings()
Retrieves the datarate dependent configuration data, the prescaler and the interdiscard.

References and Additional Information:
Reference 132457
Reference 132438

InterSampleDiscard - MAPI

- Specifies the number of consecutive samples to capture and the number of consecutive samples to discard
- Works in conjunction with preScaler to condition signals higher than the 50 Msps spec



MAPI::
Picture is showing what would happen with prescaler set to 1

References and Additional Information:
Topic 144892 PRBS Jitter Measurement by TMU
Topic 143405 Mixed Signal Lecture Series - DSP-Based Testing - Fundamentals 50 - PRBS (Pseudo RandomBinary Sequence)

MAPI: TMU Programming – Frequency Measurement

MAPI Programming 1/2:

```
TMU_TASK tskTMU;

ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(sSuite);
        cerr << "Test Suite: " << sSuite << endl;
        CONNECT();

        tskTMU.pin(m_sStartPins)
                  .setEdgeSelect(m_eStartEdgeTypeAPI)
                  .setAThreshold(m_dThresholdA)
                  .setNumMeasurements(m_iNumMeasurementArms)
                  .setNumSamples(m_iNumSamples)
                  .setInitialDiscard(m_iInitialDiscard)
                  .setNumShots(m_iNumShots);

        if(m_bUseDataRate) {
                tskTMU.pin(m_sStartPins).setDatarate(m_iDataRate, m_drSettings);
        } else {
                tskTMU.pin(m_sStartPins)
                          .setPreScaler(m_iPrescaler)
                          .setInterSampleDiscard(m_iInterDiscard);
        }
        if(m_iNumShots == 2) {
                tskTMU.pin(m_sStartPins)
                          .setBThreshold(m_dThresholdB);
        }
        if(m_bUseStopPin) {
                tskTMU.pin(m_sStopPins)
                          .setEdgeSelect(m_eStopEdgeTypeAPI)
                          .setAThreshold(m_dThresholdA)
                          .setNumMeasurements(m_iNumMeasurementArms)
                          .setNumSamples(m_iNumSamples)
                          .setInitialDiscard(m_iInitialDiscard)
                          .setNumShots(m_iNumShots);

                if(m_bUseDataRate) {
                        tskTMU.pin(m_sStopPins).setDatarate(m_iDataRate, m_drSettings);
                } else {
                        tskTMU.pin(m_sStopPins)
                                  .setPreScaler(m_iPrescaler)
                                  .setInterSampleDiscard(m_iInterDiscard);
                }
                if(m_iNumShots == 2) {
                        tskTMU.pin(m_sStopPins)
                                  .setBThreshold(m_dThresholdB);
                }
        }
```

Define TMU Task:
- Select the edge type
- Set samples
- 1 arm in pattern
- Set Voltage threshold A

Set expected data rate

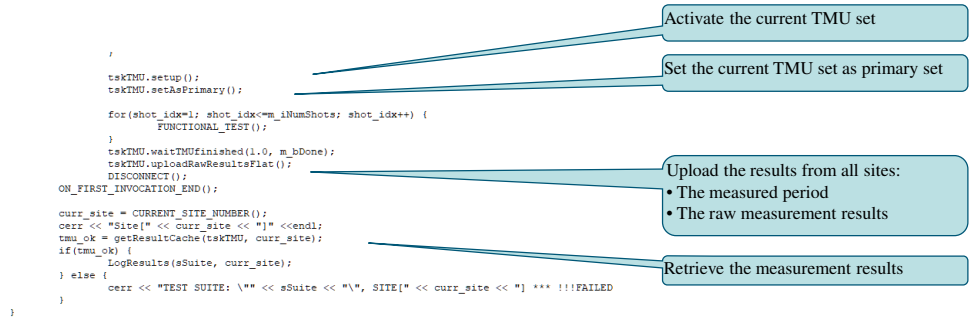
Set Voltage threshold B

If measurement requires two pins
Setup stop pin

MAPI::
Program Flow is:
Setup TMU
Run pattern and capture data by FUNCTIONAL_TEST() function.
Upload data

MAPI: TMU Programming – Frequency Measurement

MAPI Programming 2/2:




```
        ,
        tskTMU.setup();
        tskTMU.setAsPrimary();

        for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                FUNCTIONAL_TEST();
        }
        tskTMU.waitTMUfinished(1.0, m_bDone);
        tskTMU.uploadRawResultsFlat();
        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(tskTMU, curr_site);
if(tmu_ok) {
        LogResults(sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << sSuite << "\", SITE[" << curr_site << "] *** !!!FAILED
}
}
```


MAPI::
Program Flow is:
Setup TMU
Run pattern and capture data by FUNCTIONAL_TEST() function.
Upload data

LogResults() is called to calculated Frequency measurement and is same function used in RDI programming.

MAPI: TMU Programming – Frequency Measurement

```
TMU_TASK tskTMU;

ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(sSuite);
        cerr << "Test Suite: " << sSuite << endl;
        CONNECT();

        tskTMU.pin(m_sStartPins)
                  .setEdgeSelect(m_eStartEdgeTypeAPI)
                  .setAThreshold(m_dThresholdA)
                  .setNumMeasurements(m_iNumMeasurementArms)
                  .setNumSamples(m_iNumSamples)
                  .setInitialDiscard(m_iInitialDiscard)
                  .setNumShots(m_iNumShots);

        if(m_bUseDataRate) {
                tskTMU.pin(m_sStartPins).setDatarate(m_iDataRate, m_drSettings);
        } else {
                tskTMU.pin(m_sStartPins)
                          .setPreScale(m_iPrescaler)
                          .setInterSampleDiscard(m_iInterDiscard);
        }
        if(m_iNumShots == 2) {
                tskTMU.pin(m_sStartPins)
                          .setBThreshold(m_dThresholdB);
        }
        if(m_bUseStopPin) {
                tskTMU.pin(m_sStopPins)
                          .setEdgeSelect(m_eStopEdgeTypeAPI)
                          .setAThreshold(m_dThresholdA)
                          .setNumMeasurements(m_iNumMeasurementArms)
                          .setNumSamples(m_iNumSamples)
                          .setInitialDiscard(m_iInitialDiscard)
                          .setNumShots(m_iNumShots);

                if(m_bUseDataRate) {
                        tskTMU.pin(m_sStopPins).setDatarate(m_iDataRate, m_drSettings);
                } else {
                        tskTMU.pin(m_sStopPins)
                                  .setPreScaler(m_iPrescaler)
                                  .setInterSampleDiscard(m_iInterDiscard);
                }
                if(m_iNumShots == 2) {
                        tskTMU.pin(m_sStopPins)
                                  .setBThreshold(m_dThresholdB);
                }
        }
```

Test Method	tmu_ppt_tml.t_tmu_api_template.tmu_api_template
Parameters	...
Exec	
Setup	
Application Type	FREQUENCY
Number Of Measu	1
Samples Per Meas	2
DataRate	0[MHz]
Prescaler	1
Inter Sample Disc	9
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST,TMU_SP
Start Pin Edge Typ	RISE
Stop Pin	
Stop Pin Edge Typ	NONE
Limits	...

MAPI::
This slide shows an excerpt of the MAPI code in the example program. The test method parameters are shown to correlate where the MAPI inputs are coming from.
Test Method Parameters-
Application Type: FREQUENCY – This sets what calculation type will be performed on the measurements.

TMU Programming – Frequency Measurement

- Calculation of the Frequency occurs in the LogResults() function of the example code
- Function checks the selection from the parameters and calls correct calculation

```
if(m_sAppType == "PERIOD") {
    CalcPeriod(sname, site);
} else if(m_sAppType == "FREQUENCY") {
    CalcFrequency(sname, site);
} else if((m_sAppType == "DUTYCYCLE") || (m_sAppType == "PW")){
    CalcDutyCycle(sname, site);
} else if(m_sAppType == "P2P") {
    CalcPin2PinDelay(sname, site);
} else if((m_sAppType == "RISE") || (m_sAppType == "FALL")){
    CalcRiseFall(sname, site);
} else if(m_sAppType == "JITTER") {
    CalcJitter(sname, site);
} else if(m_sAppType == "EYE") {
    CalcEye(sname, site);
} else if(m_sAppType == "DRIFT") {
    CalcDrift(sname, site);
} else {
    CalcNone(sname, site);
}
```

Test Method	tmu_ppt_tml.t_tmu_rdi_template.tmu_rdi_template
Parameters	...
Exec	
Setup	
Application Type	FREQUENCY
Number Of Measu	1
Samples Per Meas	2
DataRate	0[MHz]
Prescaler	1
Inter Sample Disc	9
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST
Start Pin Edge Typ	RISE
Stop Pin	
Stop Pin Edge Typ	NONE
RDI	
Mode	BURST
Pattern Label	tmu_rdi_test_patt
Port Label	tmu_port
Vector Offset1	4
Vector Offset2	4
Limits	...

This slide shows an excerpt of the result calculation code in the example program. The LogResults selects which calculations to perform on the time stamps. Application Type: FREQUENCY – This sets what calculation type will be performed on the measurements.

TMU Programming – Frequency Measurement

- CalcFrequency() takes time stamps and performs calculations
- Calculates frequency for each start pin specified.
- Number of shots should be 1 for frequency measurement
- PostScaleResult() handles adjusting the value based upon preScaler and interSkip values

```
void tmu_base::CalcFrequency(string sname, int site)
{
        size_t pin_idx;
        int arm_idx, shot_idx, smp_idx;
        double result;
        ARRAY_D events[3];

        map<string,ARRAY_D>::iterator it;
        cerr << endl << "Calculating \"FREQUENCY\" Results for Site[" << site << "]" << endl << endl;

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(it != m_mapStartEvents[shot_idx].end()) {
                                events[shot_idx] = it->second;
                                result = 0.0;
                                for(arm_idx=0; arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                        ARRAY_D arms(m_iNumSamples);
                                        for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {
                                                arms[smp_idx] = events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                        }
                                        Wave<double> wav(arms);
                                        int len = wav.getSize();
                                        double val;
                                        val = wav.differentiate().select(1,1, len-1).mean();
                                        val = PostScaleResult(val);
                                        result += val;
                                }
                                result /= (double)m_iNumMeasurementArms;
                                JudgeAndLog(m_vecStartPins[pin_idx], "freq", 1.0/result, site);
                        } else {

                        }
                }
        }
}
```


This slide shows an excerpt of the result calculation code in the example program. The CalcFrequency function expects an edge mode of RISE or FALL. It also expects a number of shots parameter of 1 since only one TMU setup and execution is needed (Code does have a for loop for number of shots but should only go through this loop once). It calculates average of the distance between time stamps. This value is then adjusted based upon the preScaler and interSkip values from the TMU setup in the PostScaleResult(). Finally if multiple ARMS had been placed in pattern then the result average across each ARM samples is calculated. At this point we have the period result. Finally the frequency is calculated from the period.

TMU Programming – Frequency Measurement

- PostScaleResult() handles adjusting the value based upon preScaler and interSkip values
- If DataRate is specified then preScaler and interSkip (interdiscard) are read back
- If DataRate is not specified then preScaler and interSkip (interdiscard) entered in parameters are used
- Calculation

  result = val / (prescaler * (interskip+1);

```
double tmu_base::PostScaleResult(double val)
{
    double result;
    double prescale;
    double interskip;

    if(m_bUseDataRate) {
            prescale = (double)m_drSettings.prescaler;
            interskip = (double)m_drSettings.interdiscard;
    } else {
            prescale = (double)m_iPrescaler;
            interskip = (double)m_iInterDiscard;
    }
    cout << "prescale=" << prescale << endl;
    cout << "interskip=" << interskip << endl;
    result = val/(prescale*(interskip+1.0));

    return result;
}
```


This slide shows an excerpt of the result calculation code in the example program.

MAPI TMU Period Measurement

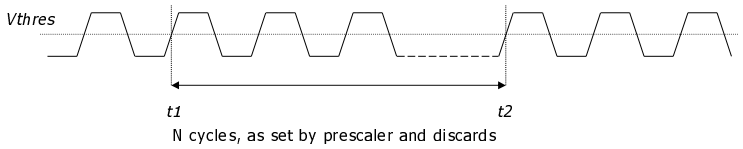
ADVANTEST

MAPI: TMU Measurements - Period

PS1600
PS9G

Measurement principle:

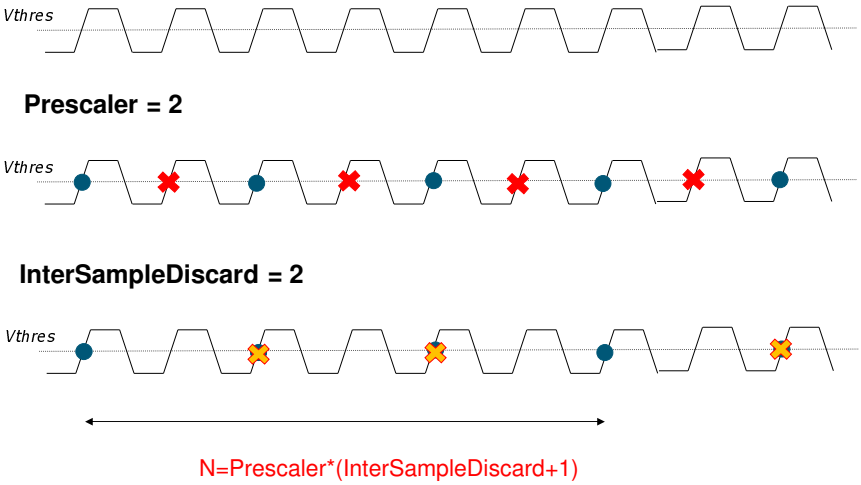
- Take two time stamps t1, t2 of the same type of event (e.g., both rising slope) which are N clock cycles apart from each other.
- Period = (t2-t1)/N
- The bigger N, the more accurate the measurement (averaging)



N cycles, as set by prescaler and discards

TMU Period Measurements – Prescaler And InterSampleDiscard

PS1600
PS9G



To measure period, it measures two same type of events t1 and t2.
Example shows setting rising slope as event.
After measured two events, upload data to DSP and calculate the period.
Setting larger N takes long but it gets more accurate measurement result.
N programs pre-scalar multiplies NB (number of events) plus one.

MAPI: TMU Programming – Period Measurement

MAPI Programming 1/2:

```
TMU_TASK tskTMU;

ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(sSuite);
        cerr << "Test Suite: " << sSuite << endl;
        CONNECT();

        tskTMU.pin(m_sStartPins)
                  .setEdgeSelect(m_eStartEdgeTypeAPI)
                  .setAThreshold(m_dThresholdA)
                  .setNumMeasurements(m_iNumMeasurementArms)
                  .setNumSamples(m_iNumSamples)
                  .setInitialDiscard(m_iInitialDiscard)
                  .setNumShots(m_iNumShots);

        if(m_bUseDataRate) {
                tskTMU.pin(m_sStartPins).setDatarate(m_iDataRate, m_drSettings);
        } else {
                tskTMU.pin(m_sStartPins)
                          .setPreScaler(m_iPrescaler)
                          .setInterSampleDiscard(m_iInterDiscard);
        }
        if(m_iNumShots == 2) {
                tskTMU.pin(m_sStartPins)
                          .setBThreshold(m_dThresholdB);
        }
        if(m_bUseStopPin) {
                tskTMU.pin(m_sStopPins)
                          .setEdgeSelect(m_eStopEdgeTypeAPI)
                          .setAThreshold(m_dThresholdA)
                          .setNumMeasurements(m_iNumMeasurementArms)
                          .setNumSamples(m_iNumSamples)
                          .setInitialDiscard(m_iInitialDiscard)
                          .setNumShots(m_iNumShots);

                if(m_bUseDataRate) {
                        tskTMU.pin(m_sStopPins).setDatarate(m_iDataRate, m_drSettings);
                } else {
                        tskTMU.pin(m_sStopPins)
                                  .setPreScaler(m_iPrescaler)
                                  .setInterSampleDiscard(m_iInterDiscard);
                }
                if(m_iNumShots == 2) {
                        tskTMU.pin(m_sStopPins)
                                  .setBThreshold(m_dThresholdB);
                }
        }
```

Define TMU Task:
- Select the edge type
- Set samples
- 1 arm in pattern
- Set Voltage threshold A

Set expected data rate

Set Voltage threshold B

If measurement requires two pins
Setup stop pin

MAPI::
Program Flow is:
Setup TMU
Run pattern and capture data by FUNCTIONAL_TEST() function.
Upload data

MAPI: TMU Programming – Period Measurement

MAPI Programming 2/2:

```
        ,

        tskTMU.setup();
        tskTMU.setAsPrimary();

        for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                FUNCTIONAL_TEST();
        }
        tskTMU.waitTMUfinished(1.0, m_bDone);
        tskTMU.uploadRawResultsFlat();
        DISCONNECT();
ON_FIRST_INVOCATION_END();

curr_site = CURRENT_SITE_NUMBER();
cerr << "Site[" << curr_site << "]" <<endl;
tmu_ok = getResultCache(tskTMU, curr_site);
if(tmu_ok) {
        LogResults(sSuite, curr_site);
} else {
        cerr << "TEST SUITE: \"" << sSuite << "\", SITE[" << curr_site << "] *** !!!FAILED
}
}
```

Activate the current TMU set

Set the current TMU set as primary set

Upload the results from all sites:
- The measured period
- The raw measurement results

Retrieve the measurement results

MAPI::
Program Flow is:
Setup TMU
Run pattern and capture data by FUNCTIONAL_TEST() function.
Upload data

LogResults() is called to calculated Frequency measurement and is same function used in RDI programming.

MAPI: TMU Programming – Period Measurement

```
TMU_TASK tskTMU;

ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(sSuite);
        cerr << "Test Suite: " << sSuite << endl;
        CONNECT();

        tskTMU.pin(m_sStartPins)
                  .setEdgeSelect(m_eStartEdgeTypeAPI)
                  .setAThreshold(m_dThresholdA)
                  .setNumMeasurements(m_iNumMeasurementArms)
                  .setNumSamples(m_iNumSamples)
                  .setInitialDiscard(m_iInitialDiscard)
                  .setNumShots(m_iNumShots);

        if(m_bUseDataRate) {
                tskTMU.pin(m_sStartPins).setDatarate(m_iDataRate, m_drSettings);
        } else {
                tskTMU.pin(m_sStartPins)
                          .setPreScale(m_iPrescaler)
                          .setInterSampleDiscard(m_iInterDiscard);
        }
        if(m_iNumShots == 2) {
                tskTMU.pin(m_sStartPins)
                          .setBThreshold(m_dThresholdB);
        }
        if(m_bUseStopPin) {
                tskTMU.pin(m_sStopPins)
                          .setEdgeSelect(m_eStopEdgeTypeAPI)
                          .setAThreshold(m_dThresholdA)
                          .setNumMeasurements(m_iNumMeasurementArms)
                          .setNumSamples(m_iNumSamples)
                          .setInitialDiscard(m_iInitialDiscard)
                          .setNumShots(m_iNumShots);

                if(m_bUseDataRate) {
                        tskTMU.pin(m_sStopPins).setDatarate(m_iDataRate, m_drSettings);
                } else {
                        tskTMU.pin(m_sStopPins)
                                  .setPreScaler(m_iPrescaler)
                                  .setInterSampleDiscard(m_iInterDiscard);
                }
                if(m_iNumShots == 2) {
                        tskTMU.pin(m_sStopPins)
                                  .setBThreshold(m_dThresholdB);
                }
        }
```

Test Method	tmu_ppt_tml.t_tmu_api_template.tmu_api_template
Parameters	...
Exec	
Setup	
Application Type	PERIOD
Number Of Measu	1
Samples Per Meas	2
DataRate	0[MHz]
Prescaler	1
Inter Sample Disc	9
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST,TMU_SP
Start Pin Edge Typ	RISE
Stop Pin	
Stop Pin Edge Typ	NONE
Limits	...

MAPI::
This slide shows an excerpt of the MAPI code in the example program. The test method parameters are shown to correlate where the MAPI inputs are coming from.
Test Method Parameters-
Application Type: PERIOD – This sets what calculation type will be performed on the measurements.

MAPI TMU Pulse Width Measurement

ADVANTEST

MAPI: TMU Programming – Pulse Width Measurement

```
TMU_TASK tskTMU;

ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(sSuite);
        cerr << "Test Suite: " << sSuite << endl;
        CONNECT();

        tskTMU.pin(m_sStartPins)
                  .setEdgeSelect(m_eStartEdgeTypeAPI)
                  .setAThreshold(m_dThresholdA)
                  .setNumMeasurements(m_iNumMeasurementArms)
                  .setNumSamples(m_iNumSamples)
                  .setInitialDiscard(m_iInitialDiscard)
                  .setNumShots(m_iNumShots);

        if(m_bUseDataRate) {
                tskTMU.pin(m_sStartPins).setDatarate(m_iDataRate, m_drSettings);
        } else {
                tskTMU.pin(m_sStartPins)
                          .setPreScale(m_iPrescaler)
                          .setInterSampleDiscard(m_iInterDiscard);
        }
        if(m_iNumShots == 2) {
                tskTMU.pin(m_sStartPins)
                          .setBThreshold(m_dThresholdB);
        }
        if(m_bUseStopPin) {
                tskTMU.pin(m_sStopPins)
                          .setEdgeSelect(m_eStopEdgeTypeAPI)
                          .setAThreshold(m_dThresholdA)
                          .setNumMeasurements(m_iNumMeasurementArms)
                          .setNumSamples(m_iNumSamples)
                          .setInitialDiscard(m_iInitialDiscard)
                          .setNumShots(m_iNumShots);

                if(m_bUseDataRate) {
                        tskTMU.pin(m_sStopPins).setDatarate(m_iDataRate, m_drSettings);
                } else {
                        tskTMU.pin(m_sStopPins)
                                  .setPreScaler(m_iPrescaler)
                                  .setInterSampleDiscard(m_iInterDiscard);
                }
                if(m_iNumShots == 2) {
                        tskTMU.pin(m_sStopPins)
                                  .setBThreshold(m_dThresholdB);
                }
        }
```

Test Method	tmu_ppt_tml.t_tmu_api_template.tmu_api_template
Parameters	...
Exec	
Setup	
Application Type	PW
Number Of Meas	1
Samples Per Mea	3
DataRate	0[MHz]
Prescaler	2
Inter Sample Disc	0
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST,TMU_SP
Start Pin Edge Typ	RISE_FALL
Stop Pin	
Stop Pin Edge Typ	NONE
Limits	...

MAPI::
This slide shows an excerpt of the MAPI code in the example program. The test method parameters are shown to correlate where the MAPI inputs are coming from.
Test Method Parameters-
Application Type: PW – This sets what calculation type will be performed on the measurements.

TMU Programming – Pulse Width Measurement

- **Pulse Width and Duty Cycle use same calculation function.**
- CalcDutyCycle () takes time stamps and performs calculations
- Calculates high pulse for each start pin specified.
- Number of shots should be 1 for Pulse Width measurement
- Edge Mode expected is RISE_FALL
- Parameters should have start pin specified

```
void tmu_base::CalcDutyCycle(string sname, int site)
{
        size_t pin_idx;
        int arm_idx, shot_idx, smp_idx;
        double result;
        double resPeriod;
        double resThi;
        double resTlo;
        ARRAY_D events[3];
        double prescale = 0.0;
        double interskip = 0.0;

        map<string,ARRAY_D>::iterator it;
        if(m_sAppType == "DUTYCYCLE") {
                cerr << endl << "Calculating \"DUTY CYCLE\" Results for Site[" << site << "]" << endl << endl;
        }
        else {
                cerr << endl << "Calculating \"PulseWidth\" Results for Site[" << site << "]" << endl << endl;
        }

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(it != m_mapStartEvents[shot_idx].end()) {
                                events[shot_idx] = it->second;
                                result = 0.0;
                                for(arm_idx=0; arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                        ARRAY_D arms(m_iNumSamples);
                                        double period=0;
                                        double thi=0;
                                        double tlo=0;

                                                                                                                ment
                                        for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {
                                                arms[smp_idx] = events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                        }

                                        // array is setup !!!
                                        if(m_bUseDataRate) {
                                                prescale = (double)m_drSettings.prescaler;
                                                interskip = (double)m_drSettings.interdiscard;
                                        } else {
                                                prescale = (double)m_iPrescaler;
                                                interskip = (double)m_iInterDiscard;
                                        }
```


Pulse Width and Duty Cycle use same calculation function because calculations compute same values.

This slide shows an excerpt of the result calculation code in the example program. The CalcDutyCycle function expects an edge mode of RISE_FALL. It also expects a number of shots parameter of 1 since only one TMU setup and execution is needed (Code does have a for loop for number of shots but should only go through this loop once). It calculates high pulse from time stamps. Finally if multiple ARMS had been placed in pattern then the result average across each ARM samples is calculated. Finally the high pulse is returned as the pulse width.

MAPI TMU Duty Cycle Measurement

ADVANTEST

MAPI: TMU Programming – Duty Cycle Measurement

```
TMU_TASK tskTMU;

ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(sSuite);
        cerr << "Test Suite: " << sSuite << endl;
        CONNECT();

        tskTMU.pin(m_sStartPins)
                  .setEdgeSelect(m_eStartEdgeTypeAPI)
                  .setAThreshold(m_dThresholdA)
                  .setNumMeasurements(m_iNumMeasurementArms)
                  .setNumSamples(m_iNumSamples)
                  .setInitialDiscard(m_iInitialDiscard)
                  .setNumShots(m_iNumShots);

        if(m_bUseDataRate) {
                tskTMU.pin(m_sStartPins).setDatarate(m_iDataRate, m_drSettings);
        } else {
                tskTMU.pin(m_sStartPins)
                          .setPreScale(m_iPrescaler)
                          .setInterSampleDiscard(m_iInterDiscard);
        }
        if(m_iNumShots == 2) {
                tskTMU.pin(m_sStartPins)
                          .setBThreshold(m_dThresholdB);
        }
        if(m_bUseStopPin) {
                tskTMU.pin(m_sStopPins)
                          .setEdgeSelect(m_eStopEdgeTypeAPI)
                          .setAThreshold(m_dThresholdA)
                          .setNumMeasurements(m_iNumMeasurementArms)
                          .setNumSamples(m_iNumSamples)
                          .setInitialDiscard(m_iInitialDiscard)
                          .setNumShots(m_iNumShots);

                if(m_bUseDataRate) {
                        tskTMU.pin(m_sStopPins).setDatarate(m_iDataRate, m_drSettings);
                } else {
                        tskTMU.pin(m_sStopPins)
                                  .setPreScaler(m_iPrescaler)
                                  .setInterSampleDiscard(m_iInterDiscard);
                }
                if(m_iNumShots == 2) {
                        tskTMU.pin(m_sStopPins)
                                  .setBThreshold(m_dThresholdB);
                }
        }
```

Test Method	tmu_ppt_tml.t_tmu_api_template.tmu_api_template
Parameters	...
Exec	
Setup	
Application Type	DUTYCYCLE
Number Of Measu	1
Samples Per Meas	3
DataRate	0[MHz]
Prescaler	2
Inter Sample Disc	0
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST,TMU_SP
Start Pin Edge Typ	RISE_FALL
Stop Pin	
Stop Pin Edge Typ	NONE
Limits	...

MAPI::
The code is same as other slides; the parameter inputs and specifying Duty Cycle is the change.
This slide shows an excerpt of the MAPI code in the example program. The test method parameters are shown to correlate where the MAPI inputs are coming from.
Test Method Parameters-
Application Type: DUTYCYCLE – This sets what calculation type will be performed on the measurements.

TMU Programming – Duty Cycle Measurement

- **Pulse Width and Duty Cycle use same calculation function.**
- CalcDutyCycle() takes time stamps and performs calculations
- Calculates period and high pulse for each start pin specified.
- Number of shots should be 1 for Pulse Width measurement
- Edge Mode expected is RISE_FALL
- Parameters should have start pin specified

```
void tmu_base::CalcDutyCycle(string sname, int site)
{
        size_t pin_idx;
        int arm_idx, shot_idx, smp_idx;
        double result;
        double resPeriod;
        double resThi;
        double resTlo;
        ARRAY_D events[3];
        double prescale = 0.0;
        double interskip = 0.0;

        map<string,ARRAY_D>::iterator it;
        if(m_sAppType == "DUTYCYCLE") {
                cerr << endl << "Calculating \"DUTY CYCLE\" Results for Site[" << site << "]" << endl << endl;
        }
        else {
                cerr << endl << "Calculating \"PulseWidth\" Results for Site[" << site << "]" << endl << endl;
        }

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(it != m_mapStartEvents[shot_idx].end()) {
                                events[shot_idx] = it->second;
                                result = 0.0;
                                for(arm_idx=0; arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                        ARRAY_D arms(m_iNumSamples);
                                        double period=0;
                                        double thi=0;
                                        double tlo=0;

                                        for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {
                                                arms[smp_idx] = events[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                        }

                                        // array is setup !!!
                                        if(m_bUseDataRate) {
                                                prescale = (double)m_drSettings.prescaler;
                                                interskip = (double)m_drSettings.interdiscard;
                                        } else {
                                                prescale = (double)m_iPrescaler;
                                                interskip = (double)m_iInterDiscard;
                                        }
}
```


Pulse Width and Duty Cycle use same calculation function because calculations compute same values.

This slide shows an excerpt of the result calculation code in the example program. The CalcDutyCycle function expects an edge mode of RISE_FALL. It also expects a number of shots parameter of 1 since only one TMU setup and execution is needed (Code does have a for loop for number of shots but should only go through this loop once). It calculates period and high pulse from time stamps. Finally if multiple ARMS had been placed in pattern then the result average across each ARM samples is calculated. Finally the duty cycle is calculated by dividing the high pulse time by the period and multiplied by 100.

MAPI TMU Pin to Pin Delay Measurement

ADVANTEST

MAPI: TMU Programming – Pin To Pin Delay Measurement

```
TMU_TASK tskTMU;

ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(sSuite);
        cerr << "Test Suite: " << sSuite << endl;
        CONNECT();

        tskTMU.pin(m_sStartPins)
                  .setEdgeSelect(m_eStartEdgeTypeAPI)
                  .setAThreshold(m_dThresholdA)
                  .setNumMeasurements(m_iNumMeasurementArms)
                  .setNumSamples(m_iNumSamples)
                  .setInitialDiscard(m_iInitialDiscard)
                  .setNumShots(m_iNumShots);

        if(m_bUseDataRate) {
                tskTMU.pin(m_sStartPins).setDatarate(m_iDataRate, m_drSettings);
        } else {
                tskTMU.pin(m_sStartPins)
                          .setPreScale(m_iPrescaler)
                          .setInterSampleDiscard(m_iInterDiscard);
        }
        if(m_iNumShots == 2) {
                tskTMU.pin(m_sStartPins)
                          .setBThreshold(m_dThresholdB);
        }
        if(m_bUseStopPin) {
                tskTMU.pin(m_sStopPins)
                          .setEdgeSelect(m_eStopEdgeTypeAPI)
                          .setAThreshold(m_dThresholdA)
                          .setNumMeasurements(m_iNumMeasurementArms)
                          .setNumSamples(m_iNumSamples)
                          .setInitialDiscard(m_iInitialDiscard)
                          .setNumShots(m_iNumShots);

                if(m_bUseDataRate) {
                        tskTMU.pin(m_sStopPins).setDatarate(m_iDataRate, m_drSettings);
                } else {
                        tskTMU.pin(m_sStopPins)
                                  .setPreScaler(m_iPrescaler)
                                  .setInterSampleDiscard(m_iInterDiscard);
                }
                if(m_iNumShots == 2) {
                        tskTMU.pin(m_sStopPins)
                                  .setBThreshold(m_dThresholdB);
                }
        }
```

Test Method	tmu_ppt_tml.t_tmu_api_template.tmu_api_template
Parameters	...
Exec	
Setup	
Application Type	P2P
Number Of Meas	1
Samples Per Mea	10
DataRate	25[MHz]
Prescaler	1
Inter Sample Disc	0
Initial Discard	0
Number Of Shots	1
Threshold A	0.75[V]
Threshold B	0[V]
Pins	
Start Pin	TMU_ST
Start Pin Edge Typ	RISE
Stop Pin	TMU_SP
Stop Pin Edge Typ	FALL
Limits	...

MAPI::
The code is same as other slides; the parameter inputs and specifying P2P is the change. Measuring difference between the rising edge of the start pin to the fall edge of the stop pin.
This slide shows an excerpt of the MAPI code in the example program. The test method parameters are shown to correlate where the MAPI inputs are coming from.
Test Method Parameters-
Application Type: P2P – This sets what calculation type will be performed on the measurements.

TMU Programming – Pin to Pin Delay Measurement

- CalcPin2PinDelay() takes time stamps and performs calculations
- Requires a start pin and a stop pin to be specified in the parameters
- Calculates delta between Stop pin time stamps and the Start pin time stamps
- Number of shots should be 1 for Pin to pin Delay measurement
- Edge Mode expected is RISE or FALL and can be different between the start and stop pin

```
void tmu_base::CalcPin2PinDelay(string sname, int site)
{
        size_t pin_idx;
        int arm_idx, shot_idx, smp_idx;
        double result;
        ARRAY_D events1[3];
        ARRAY_D events2[3];
        map<string,ARRAY_D>::iterator it1;
        map<string,ARRAY_D>::iterator it2;
        cerr << endl << "Calculating \"PIN TO PIN DELAY\" Results for Site[" << site << "]" << endl << endl;

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it1 = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        it2 = m_mapStopEvents[shot_idx].find(m_vecStopPins[pin_idx]);
                        if(it1 != m_mapStartEvents[shot_idx].end() && it2 != m_mapStopEvents[shot_idx].end()) {
                                events1[shot_idx] = it1->second;
                                events2[shot_idx] = it2->second;
                                result = 0.0;
                                for(arm_idx=0; arm_idx<m_iNumMeasurementArms; arm_idx++) {
                                        ARRAY_D arms1(m_iNumSamples);
                                        ARRAY_D arms2(m_iNumSamples);
                                        for(smp_idx=0; smp_idx<m_iNumSamples; smp_idx++) {
                                                arms1[smp_idx] = events1[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                                arms2[smp_idx] = events2[shot_idx][(arm_idx*m_iNumSamples)+smp_idx];
                                                result += (arms2[smp_idx]-arms1[smp_idx]);
                                        }
                                        result /= m_iNumSamples;
                                }
                                result /= (double)m_iNumMeasurementArms;

                                //cout << "Pin2PinDelay=" << result << endl;
                                JudgeAndLog(m_vecStopPins[pin_idx], "pin2pin", result, site);
                        } else {

                        }
                }
        }
}
```


This slide shows an excerpt of the result calculation code in the example program. The CalcPin2PinDelay function expects a number of shots parameter of 1 since only one TMU setup and execution is needed (Code does have a for loop for number of shots but should only go through this loop once). It delta between the stop pin and start pin time stamps. The delta is average across the number of samples. Finally if multiple ARMS had been placed in pattern then the result average across each ARM samples is calculated. The delta result between the 2 pins is returned as the result.

Code is similar to the CalcPulseWidth()

MAPI TMU Rise/Fall Time Measurement

ADVANTEST

MAPI: TMU Programming – Rise/Fall Time Measurement

```
TMU_TASK tskTMU;

ON_FIRST_INVOCATION_BEGIN();
        GET_TESTSUITE_NAME(sSuite);
        cerr << "Test Suite: " << sSuite << endl;
        CONNECT();

        tskTMU.pin(m_sStartPins)
                  .setEdgeSelect(m_eStartEdgeTypeAPI)
                  .setAThreshold(m_dThresholdA)
                  .setNumMeasurements(m_iNumMeasurementArms)
                  .setNumSamples(m_iNumSamples)
                  .setInitialDiscard(m_iInitialDiscard)
                  .setNumShots(m_iNumShots);

        if(m_bUseDataRate) {
                tskTMU.pin(m_sStartPins).setDatarate(m_iDataRate, m_drSettings);
        } else {
                tskTMU.pin(m_sStartPins)
                          .setPreScale(m_iPrescaler)
                          .setInterSampleDiscard(m_iInterDiscard);
        }
        if(m_iNumShots == 2) {
                tskTMU.pin(m_sStartPins)
                          .setBThreshold(m_dThresholdB);
        }
        if(m_bUseStopPin) {
                tskTMU.pin(m_sStopPins)
                          .setEdgeSelect(m_eStopEdgeTypeAPI)
                          .setAThreshold(m_dThresholdA)
                          .setNumMeasurements(m_iNumMeasurementArms)
                          .setNumSamples(m_iNumSamples)
                          .setInitialDiscard(m_iInitialDiscard)
                          .setNumShots(m_iNumShots);

                if(m_bUseDataRate) {
                        tskTMU.pin(m_sStopPins).setDatarate(m_iDataRate, m_drSettings);
                } else {
                        tskTMU.pin(m_sStopPins)
                                  .setPreScaler(m_iPrescaler)
                                  .setInterSampleDiscard(m_iInterDiscard);
                }
                if(m_iNumShots == 2) {
                        tskTMU.pin(m_sStopPins)
                                  .setBThreshold(m_dThresholdB);
                }
        }
```

Second Shot requires voltage Threshold B to be set up

Test Method	tmu_ppt_tml.t_tmu_api_template.tmu_api_template
Parameters	...
Exec	
Setup	
Application Type	RISE
Number Of Meas	1
Samples Per Meas	10
DataRate	25[MHz]
Prescaler	1
Inter Sample Disc	0
Initial Discard	0
Number Of Shots	2
Threshold A	0.75[V]
Threshold B	2.25[V]
Pins	
Start Pin	TMU_ST,TMU_SP
Start Pin Edge Typ	RISE
Stop Pin	
Stop Pin Edge Typ	NONE
Limits	...

MAPI::
The code is same as other slides; the parameter inputs and specifying RISE is the change. Measuring rise time of the start from voltage threshold A to voltage threshold B.
This slide shows an excerpt of the MAPI code in the example program. The test method parameters are shown to correlate where the MAPI inputs are coming from.
Test Method Parameters-
Application Type: RISE – This sets what calculation type will be performed on the measurements.

FALL time measurement works same just by specifying FALL and swapping voltage thresholds.

TMU Programming – Rise/Fall Time Measurement

- CalcRiseFall() takes time stamps and performs calculations
- Requires Voltage Threshold A and B to be specified in the parameters
- Calculates delta between Stop pin time stamps and the Start pin time stamps
- Number of shots should be 2 fo Rise/Fall time measurement fo each voltage threshold
- Edge Mode expected is RISE o FALL

```
void tmu_base::CalcRiseFall(string sname, int site)
{
        size_t pin_idx;
        int shot_idx;
        double result;
        ARRAY_D events[3];

        map<string,ARRAY_D>::iterator it;
        cerr << endl << "Calculating \"RISE/FALL\" Results for Site[" << site << "]" << endl << endl;

        for(pin_idx=0; pin_idx<m_vecStartPins.size(); pin_idx++) {
                for(shot_idx=1; shot_idx<=m_iNumShots; shot_idx++) {
                        it = m_mapStartEvents[shot_idx].find(m_vecStartPins[pin_idx]);
                        if(it != m_mapStartEvents[shot_idx].end()) {
                                events[shot_idx] = it->second;
                        }
                }

                events[0].resize(m_iNumMeasurementArms*m_iNumSamples);

                for(int smp_idx=0; smp_idx<events[0].size(); smp_idx++) {
                        events[0][smp_idx] = events[2][smp_idx] - events[1][smp_idx];
                }
                Wave<double> wav(events[0]);
                result = wav.mean();
                //cout << "RISE/FALL=" << (result) << endl;
                if(m_eStartEdgeTypeAPI == TMU::RISE )
                        JudgeAndLog(m_vecStartPins[pin_idx], "rise", result, site);
                if(m_eStartEdgeTypeAPI == TMU::FALL )
                        JudgeAndLog(m_vecStartPins[pin_idx], "fall", result, site);

        }
}
```


This slide shows an excerpt of the result calculation code in the example program. The CalcRiseFall function expects a number of shots parameter of 2 since a TMU setup and execution is needed for both the voltage threshold A and the voltage threshold B. The delta between the voltage threshold time stamps is calculated. The delta is average across the number of samples.

Additional Test Types

ADVANTEST

Additional TMU Test Types

- Average
- PLL Lock Time
- Jitter

TMU can be used to perform these additional tests types.

TMU Average Measurement

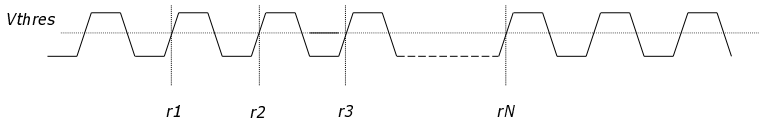
ADVANTEST

MAPI: TMU Measurements - Average

PS1600
PS9G

Measurement principle:

- Capture edges for N samples
- Average = (Sum of edge times) / N
- The bigger N, the more accurate the measurement (averaging)



Calculations from genericTMU

TMU PLL Lock Time Measurement

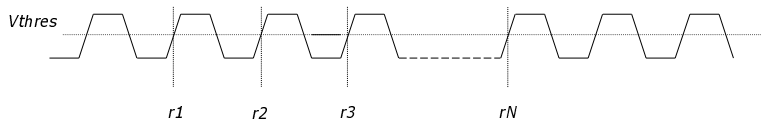
ADVANTEST

MAPI: TMU Measurements - Average

PS1600
PS9G

Measurement principle:

- Capture edges for N samples
- Calculate frequency between each captured edge
- When calculated frequency is
  - > expected frequency * Pll Tolerance % and
  - < expected frequency / PLL tolerance %
  - Return the edge time



Calculations from genericTMU

TMU Jitter Measurement

ADVANTEST

Jitter Measurement

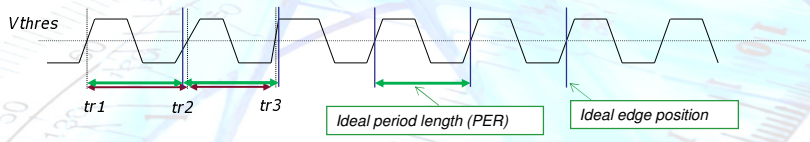
PS1600
PS9G

TMU Measurements – Period Jitter

Measurement/Calculation principle:

- Determine the ideal period length PER based on signal frequency
- Calculate a jitter value for second transition (tr2) as: Jitter = (tr2_value - tr1_value) - PER
- Calculate a jitter value for third transition (tr3) as: Jitter = (tr3_value - tr2_value) - PER
- The basic formula is: Jitter = (tri_value - tr(i-1)_value) - PER
  where i is a number of samples taken between 2 and N

- TMU can measure both period jitter and total jitter, based on a kind of calculation, since it measures absolute values for every transition



MAPI: Jitter Measurement

PS600
PS9

TMU Programming – Period Jitter Measurement

```
48 TMU_TASK task("setA");
49 TMU_RESULTS tmuResults;
50 TMU::DATARATE_DEPENDENT_VALUES_TYPE settings;
51
52 ON_FIRST_INVOCATION_BEGIN();
53    task.pin("rcv1")
54        .setEdgeSelect(TMU::RISE)
55        .setDatarate(mDataRate*1E6, settings)
56        .setNumSamples(mNumberOfSamples)
57        .setNumMeasurements(1);
58
59    task.setAsPrimary();
60    task.setup();
61    FUNCTIONAL_TEST() ;
62
63    task.pin(" rcv1"    ).uploadPeriodResults(tmuResults);
64 ON_FIRST_INVOCATION_END();
65
66 DOUBLE period_jitter_peak_peak = tmuResults.getPeriodPP("rcv1")     * 1E9;
67 DOUBLE period_jitter_rms       = tmuResults.getPeriodStddev("rcv1") * 1E9;
```

Define TMU Task:
- Select the edge type
- Set expected data rate
- Set number of sample to be taken

Set the current TMU set as primary set

Activate the current TMU set

Upload the results from all sites:
- The calculated measurement results

Retrieve the measurement results and convert to [ps]

Here is another example of the TMU test method library programming code for Jitter measurement.
Program Flow is:
Setup TMU
Run pattern and capture data by FUNCTIONAL_TEST() function.
Upload data
Run DSP functions for jitter calculation

Jitter Measurement

PS600
PS9

TMU Measurements – Total Jitter

Measurement principle:

Calculate ideal edge positions based on signal's frequency

Take time stamps of N rising and/or falling transitions

Calculate the difference between ideal and measured position. This is jitter.

Repeat it for all taken samples. This will give you total jitter.

Calculation of total jitter:

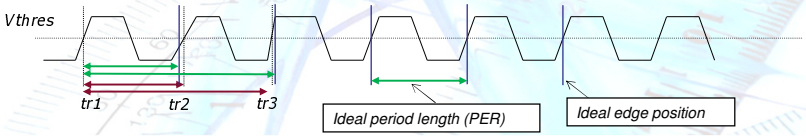
Calculate a period length PER based on signal frequency

Calculate a jitter value for second transition (tr2) as: Jitter = (tr2_value - tr1_value) - PER

Calculate a jitter value for third transition (tr3) as: Jitter = (tr3_value - tr1_value) - 2*PER

The basic formula is: Jitter = (tri_value-tr1_value) - (i-1)*PER

where i is a number of samples taken between 2 and N

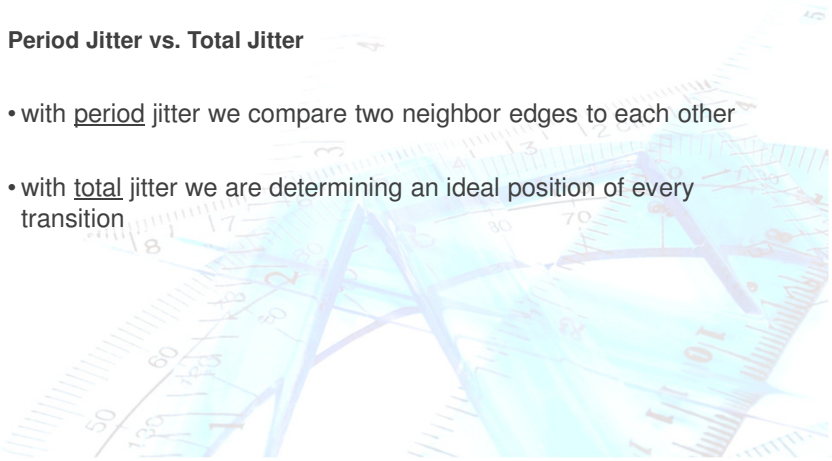


Jitter Measurement

PS600
PS9

Period Jitter vs. Total Jitter

- with period jitter we compare two neighbor edges to each other
- with total jitter we are determining an ideal position of every transition



END

RDI::